

高速缓存

学习目标

学习本章后，你将可以：

- 概括描述计算机存储系统的主要特点和存储器层次结构的使用情况。
- 描述高速缓存的基本概念和意图。
- 讨论高速缓存设计的关键元素。
- 区分直接映射、相联映射和组相联映射。
- 解释使用多层高速缓存的原因。
- 理解多层存储器对性能的影响。

虽然概念上看似简单，但计算机存储器展示的类型、技术、组成、性能和成本等方面可能是计算机系统中最广泛的。在满足计算机系统存储需求方面，没有一种技术是最优的。因此，典型计算机系统配备的是存储器子系统的多级结构，有些是系统内部的（存储器直接访问），有些是系统外部的（处理器通过 I/O 模块访问）。

本章和下一章关注的是内部存储元素，第6章专门讨论外部存储器。首先，第一节讨论计算机存储器的关键特性。本章剩余部分将讨论所有现代计算机系统的一个基本元素：高速缓存。

4.1 计算机存储系统概述

4.1.1 存储系统的特性

如果我们根据存储系统的关键特性对其进行分类，那么计算机存储器这一复杂的主题就会变得更加易于管理。表4-1列出了最重要的关键特性。

表4-1 计算机存储系统的关键特性

位置	内部（如：处理器寄存器、高速缓存、主存）	性能	访问时间
	外部（如：光盘、磁盘、磁带）		周期时间
容量	字数		传输速率
	字节数	物理类型	半导体
传输单位	字		磁性
	块		光学的
访问方式	顺序		磁光
	直接	物理特性	易失性 / 非易失性
	随机		可擦除 / 不可擦除
	相联	组成	存储模块

表4-1中的术语“位置”指的是存储器在计算机的内部还是外部。内部存储器通常等同于主存，但是内部存储器也有其他形式。处理器需要自己的本地存储器，其形式就是寄存器

(见图 2-3)。此外,正如我们所见,处理器的控制单元部分可能也想要自己的内部存储器。后两种内部存储器将在后续章节中讨论。高速缓存是另一种内部存储器。外部存储器由外围存储设备组成,如磁盘和磁带,处理器通过 I/O 模块访问它们。

存储器的一个显著特征是它的容量。对内部存储器来说,通常是用字节(1 字节=8 位)或字来表示。常见字长为 8 位、16 位和 32 位。外部存储器容量一般用字节表示。

一个相关的概念是**传输单位**。对内部存储器而言,传输单位等于进出存储器模块的电线条数。它可能等于字长,但通常会更大一些,比如 64 位、128 位或 256 位。为了阐明这一点,需考虑内部存储器的三个相关概念:

- **字** 这是存储器组织的“天然”单位。字的大小一般等于表示一个整数或指令长度的位数。可惜的是,存在一些例外。例如,CRAY C90(一种较老的 CRAY 超级计算机)字长 64 位,但用 46 位表示整数。Intel x86 架构有多种指令长度,表示为字节的倍数,但其字长为 32 位。
- **可寻址单元** 在有些系统中,可寻址单元是字。但是,也有一些系统允许在字节级别上寻址。不管是哪种情况,地址位数 A 与可寻址单元个数之间的关系是 $2^A=N$ 。
- **传输单位** 对主存来说,这是一次读出或写入存储器的位数。传输单位不需要等于一个字或一个可寻址单元。对外部存储器来说,数据通常以比一个字大得多的单位来传输,它们被称为块。

存储器类型之间的另一个区别是**访问数据单元的方式**,它们包括:

- **顺序访问** 存储器是按照数据单元来组织的,称为记录。必须以特定的线性顺序来访问。存储的寻址信息用于分离记录并协助检索过程。使用了共享读写机制,必须从当前位置移动到目标位置,经过并不访问每个中间记录。因此,对任意记录的访问时间是高度变化的。第 6 章讨论的磁带单元就是顺序访问。
- **直接访问** 和顺序访问一样,直接访问页涉及共享读写机制。但是,单个块或记录有基于物理位置的唯一地址。访问是通过直接访问一般邻近区域以及顺序搜索、计数或等待达到最终位置来完成的。同样,访问时间也是变化的。第 6 章讨论的磁盘单元就是直接访问。
- **随机访问** 存储器中每个可寻址位置都是唯一的,物理固定的寻址机制。对给定位置的访问时间是恒定的,与之前的访问顺序无关。因此,可以随机选择任何位置并直接寻址和访问。主存和某些高速缓存系统就是随机访问。
- **相联** 这是随机访问类型的存储器,允许用户对指定匹配的字内所需位位置进行比较,且所有字的操作是同时进行的。因此,对一个字的检索是基于其部分内容,而不是其地址。与普通随机访问存储器一样,每个位置都有自己的寻址机制,检索时间是恒定的,与位置或之前的访问模式无关。高速缓存可以使用相联访问。

从用户角度来看,存储器最重要的两个特性是容量和性能。性能使用了三个参数:

- **访问时间(延迟)** 对随机访问存储器而言,访问时间是其执行读或写操作所消耗的时间,即从地址呈现给存储器到数据已经存储或可供使用的时间间隔。对非随机访问存储器而言,访问时间是将读写机制定位到所需位置的时间。
- **存储器周期时间** 这个概念主要应用于随机访问存储器,它包含了访问时间和第二次访问开始前所需的所有额外时间。如果瞬态在信号线上消失,或重新生成被破坏性读取的数据,就需要额外的时间。注意,存储器周期时间与系统总线有关,与处理器无关。

- **传输速率** 这是数据传输到存储器单元或从存储器单元传输出去的速率，等于 $1/\text{周期时间}$ 。对非随机访问存储器而言，如下关系成立：

$$T_n = T_A + \frac{n}{R} \quad (4.1)$$

其中：

T_n = 读或写 n 位的平均时间

T_A = 平均访问时间

n = 位数

R = 传输速率，以位数 / 秒 (bps) 为单位

存储器采用了各种**物理类型**。目前最常见的是半导体存储器、磁性表面存储器、用于磁盘和磁带，以及光学的和磁光的存储器。

数据存储的几个**物理特性**很重要。在易失性存储器中，信息在断电时会自然衰减或丢失。在非易失性存储器中，除非是故意更改，否则信息一旦被记录就会维持不变，不需要电力来保留信息。磁性表面存储器是非易失性的。半导体存储器（集成电路上的存储器）可能是易失性的也可能是非易失性的。不可擦除的存储器不能更改，除非是破坏存储单元。这种类型的半导体存储器被称为只读存储器（ROM）。当然，一个实用的不可擦除存储器也必须是非易失性的。

对随机访问存储器来说，**组织**是关键的设计问题。在这里，组织是指组成字的位的物理排列。正如第5章所述，这种显而易见的排列并不总是被使用。

4.1.2 存储器层次结构

计算机存储器的设计限制可以归结为三个问题：多大？多快？多贵？

“多大”的问题还没有定论。如果给出了容量，可能会开发应用程序来使用它。“多快”的问题在某种意义上更容易回答。要获得更高的性能，存储器必须能跟上处理器的速度。也就是说，当处理器在执行指令时，我们不希望它被迫停下来等待指令或操作数。最后一个问题也是需要考虑的。对实际系统来说，相对于其他组件，存储器的成本必须是合理的。

正如预期的一样，存储器的三个关键特性之间存在权衡：容量、访问时间和成本。实现存储系统使用了各种各样的技术，在这些技术中存在如下关系：

- 访问时间越快，每位成本越高。
- 容量越大，每位成本越低。
- 容量越大，访问时间越长。

设计人员面临的困境是显而易见的。设计人员希望使用提供大容量存储器的存储器技术，不仅因为需要容量，还因为每位成本较低。但是，为了满足性能需求，设计人员又需要使用昂贵的、容量相对较小的、访问时间较短的存储器。

这个困境的出路在于不要依赖于单一的存储器组件或技术，而是利用**存储器层次结构**。典型的层次结构如图4-1所示。当沿着这个层次结构向下走的时候，会发生下列情况：

- 降低每位成本。
- 增加容量。
- 增加访问时间。
- 降低处理器访问该存储器的频率。

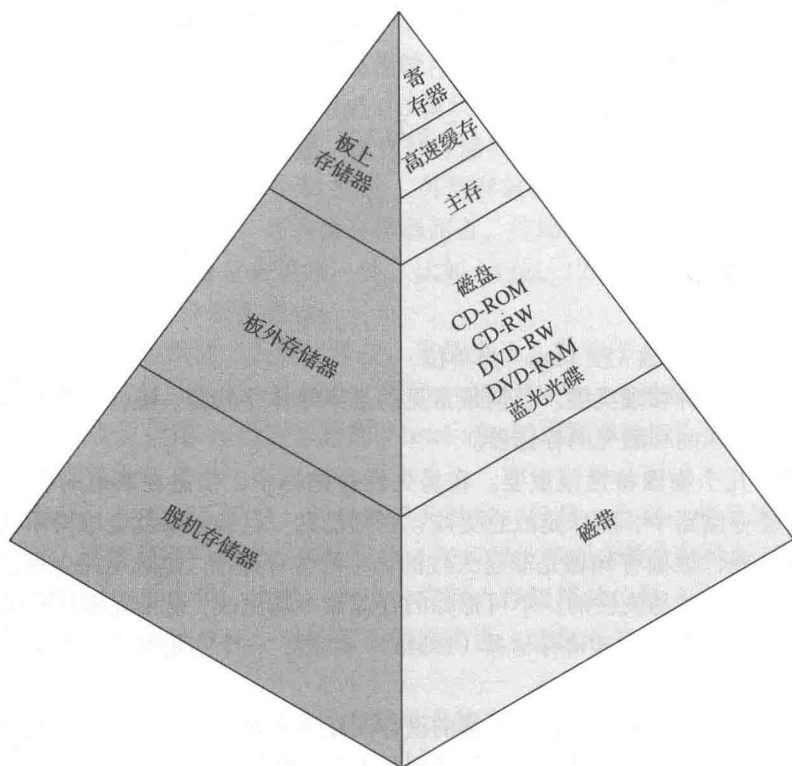


图 4-1 存储器层次结构

因此，更小、更贵、更快的存储器由更大、更便宜、更慢的存储器补充。这种组织结构成功的关键是 d 项：降低访问频率。本章稍后讨论高速缓存，以及第 8 章讨论虚存时将更加详细地探讨这个概念。现在只做一个简要的解释。

使用两级存储器来减少平均访问时间是可行的，但是必须是 a 到 d 条件适用才行。通过采用各种技术，存在一系列满足条件 a 到 c 的存储系统。幸运的是，条件 d 一般也有效。

条件 d 有效的基础是被称为访问局部性的原理 [DENN68]。在一个程序执行期间，不论是指令还是数据，处理器对内存的访问往往比较集中。程序通常包括许多迭代循环和子例程。一旦进入循环或子例程，就会重复引用一小部分指令。同样，对表格或数组的操作涉及访问数据字集。在较长时间里，使用的集会发生变化，但是在较短时间内，处理器主要处理固定集的存储器访问。

例 4.1 设处理器访问两级存储器。L1 包含 1000 个字，访问时间为 $0.01 \mu\text{s}$ ；L2 包含 100 000 个字，访问时间为 $0.1 \mu\text{s}$ 。如果被访问的数据在 L1，那么处理器直接访问该数据。如果数据在 L2，那么它首先被传输到 L1，然后再被处理器访问。为了简单起见，我们忽略处理器确定字在 L1 还是在 L2 所需的时间。图 4-2 展示了这种情况下曲线的大致形状。这个图把两级存储器的平均访问时间表示为命中率 H 的函数，这里的 H 定义为在较快存储器（如：高速缓存）中找到数据的全部存储器访问所占的比例， T_1 是 L1 的访问时间， T_2 是 L2 的访问时间^①。可以看出，对高百分比的 L1 访问来说，其平均总访问时间更接近 L1 的访

① 如果所访问的字在较快的存储器中被发现，这种情况被定义为命中。如果所访问的字没有在较快的存储器中被发现，则发生缺失。

问时间而不是 L2 的访问时间。

在我们的例子中，假设 95% 的存储器访问都在 L1 中得到满足。那么访问一个字的平均时间可以表示为

$$(0.95)(0.01\mu\text{s}) + (0.05)(0.01\mu\text{s} + 0.1\mu\text{s}) \\ = 0.0095 + 0.0055 = 0.015\mu\text{s}$$

和预期的一样，该平均访问时间非常接近于 0.01 μs ，而不是接近于 0.1 μs 。

因此，可以在这个层次结构上组织数据，使得对较低层次的访问百分比大大低于对其上一层的访问百分比。考虑上例给出的两级存储器。让 L2 存储器包含全部程序指令和数据，而当前使用的指令和数据集则可以暂时存放在 L1 中。有时，L1 中的集需要交换回 L2，以便为进入 L1 的新集腾出空间。但是，平均而言，大多数引用都指向 L1 中包含的指令和数据。

这个原则可以应用于两级以上的存储器，如图 4-1 所示的存储器层次结构。最快、最小、最贵的存储器类型包括了处理器内部的寄存器。通常，处理器包含有十几个这样的寄存器，也有些机器包含了数百个寄存器。主存是计算机主要的内部存储系统。主存中的每个位置都有唯一的地址。主存一般用更快、更小的高速缓存来进行扩展。高速缓存通常对程序员，或者实际上对处理器是不可见的。它是一种执行主存与处理器寄存器之间数据移动的设备，以提高性能。

上面描述的三种形式的存储器通常是易失性的，采用半导体技术。三层结构的使用利用了半导体存储器有多种类型的事实，它们在速度和成本上不一样。数据在外部大容量存储器上保存得更为持久，其中最常见的是硬盘和可移动介质，比如可移动磁盘、磁带和光盘存储器。外部非易失性存储器也称为**辅助存储器**（secondary memory 或 auxiliary memory）。它们用于保存程序和数据文件，一般仅在文件和记录层面对程序员可见，而不是单个字节或字。磁盘也用于提供对主存的扩展，称为虚拟内存，详细讨论参见第 8 章。

其他形式的存储器也可能包含在层次结构中。例如，IBM 大型主机包含一种称为扩展存储器的内部存储器。它使用半导体技术，比主存更慢、更便宜。严格地说，这种存储器不适合层次结构，但它是一种分支：数据可以在主存和扩展存储器之间移动，但不能在扩展存储器和外存之间移动。其他形式的辅助存储器包括了光盘和磁光盘。最后，可以在软件中把其他层次有效地添加到层次结构中。主存的一部分可以被用作缓冲区，暂时保存要读出到磁盘的数据。这种技术有时被称为**磁盘缓存**^①，它从两个方面提高了性能：

- 磁盘写入具有簇聚性。不同于许多少量数据的传输，这里有的是几个大量数据的传输。这提高了磁盘性能，并最大限度地减少了处理器的参与。
- 有些被写出的数据在下一次转储到磁盘之前可能会被程序引用。在这种情况下，数据会快速地从软件缓存中检索，而不是缓慢地从磁盘中检索。

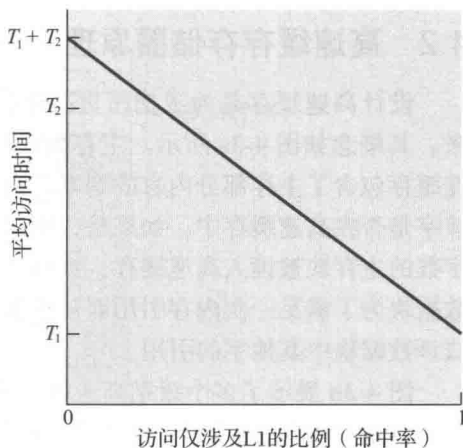


图 4-2 仅涉及 L1 访问的性能（命中率）

① 磁盘缓存通常是一种纯软件技术，本书不加讨论。相关讨论参见 [STAL15]。

附录 4A 讨论了多层存储器结构对性能的影响。

4.2 高速缓存存储器原理

设计高速缓存是为了把昂贵高速存储器的访问时间与便宜低速存储器的大容量结合起来。其概念如图 4-3a 所示。主存相对容量较大速度较慢，高速缓存容量较小速度较快。高速缓存包含了主存部分内容的副本。当处理器尝试从内存读取一个字时，要进行检查以确定该字是否在高速缓存中。如果是，则这个字被传输给处理器。如果不是，那么一个包含固定字数的主存块被读入高速缓存，然后该字被传输给处理器。由于访问的局部性现象，当一个数据块为了满足一次内存引用而被获取到高速缓存时，很有可能将来也会有对同一内存位置或该数据块中其他字的引用。

图 4-3b 展示了多个级别高速缓存的使用。与 L1 高速缓存相比，L2 高速缓存速度更慢，但其容量一般要更大；与 L2 高速缓存相比，L3 高速缓存速度更慢，但其容量一般要更大。

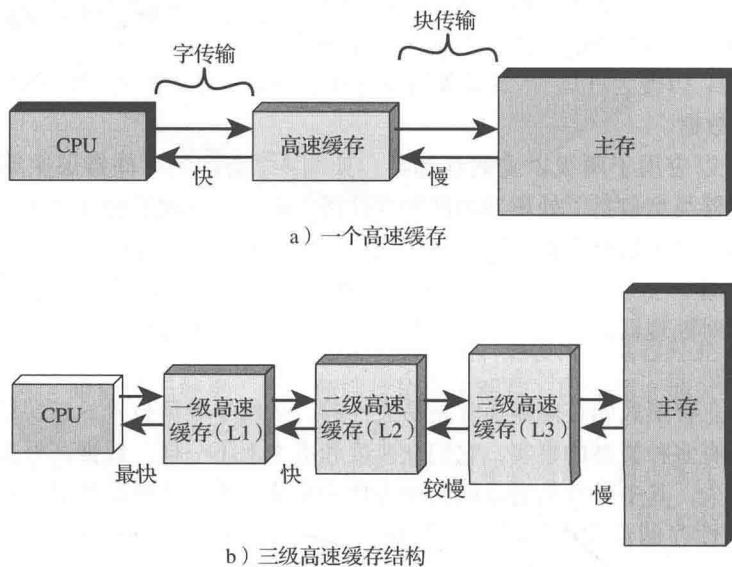


图 4-3 高速缓存和主存

图 4-4 展示高速缓存 / 主存系统的结构。主存包含多达 2^n 个可寻址字，每个字有唯一的 n 位地址。为了映射，把内存看作由许多固定长度的块构成，每个块包含 K 个字。也就是说，主存有 $M=2^n/K$ 个块。高速缓存由 m 个块构成，每个块称为一个行^①。每个行包含 K 个字，以及一个由若干位组成的标签。每个行还包括有控制位（图中未显示），比如，用一个位指示该行自被加载到高速缓存后是否被修改过。一个行的长度，不包括标签和控制位，被称为行的**大小**。行的**大小**可以小到 32 位，其中每个“字”都是一个字节；在这种情况下，行大小为 4 字节。行数大大小于主存块数 ($m \ll M$)。任何时候，主存块的某些子集都驻留在高速缓存的行中。如果主存块中的一个字被读取，这个块就会被传送到高速缓存的一个行上。由于块数大于行数，每个行都不会唯一且永久的专用于一个特定的块。因此，每个行都有一个标签来

① 在提到高速缓存的基本单位时，使用术语“行”，而不是术语“块”，原因有两个：1) 避免与主存块相混淆，一个主存块所包含的数据字的个数与一个高速缓存行的相同；2) 一个高速缓存行不仅仅只有 K 个字的数据，主存块就是这样，高速缓存行还有标签位和控制位。

指明当前存放的是哪个特定块。标签通常是主存地址的一部分，本节稍后将进行讨论。

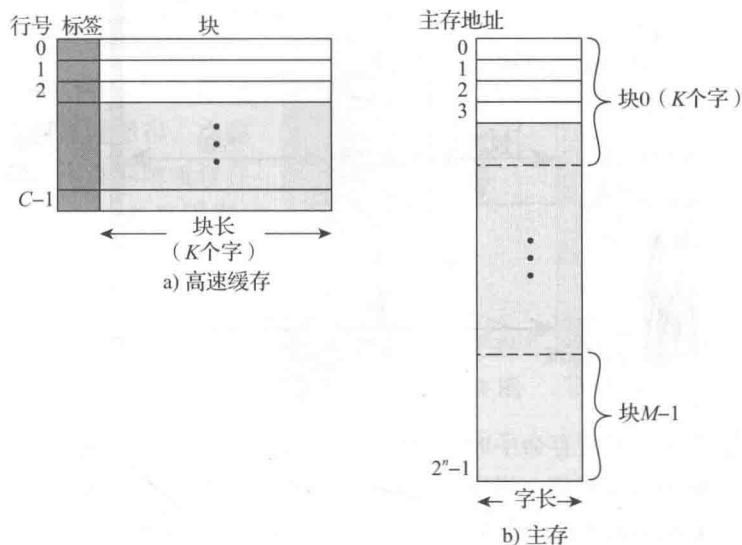


图 4-4 高速缓存 / 主存结构

图 4-5 演示了读操作。处理器生成被读取字的读地址 (RA)。如果这个字已经在高速缓存中，它就被传送给处理器。否则，包含这个字的主存块就会被加载到高速缓存，这个字则被传送给处理器。图 4-5 表明后两个操作是同时发生的，并影响到了图 4-6 所示的结构，图 4-6 给出的是当代高速缓存的典型结构。在这种结构中，高速缓存通过数据线、控制线和地址线与处理器相连。数据线和地址线还连接数据和地址缓冲区，这些缓冲区又连接到通往主

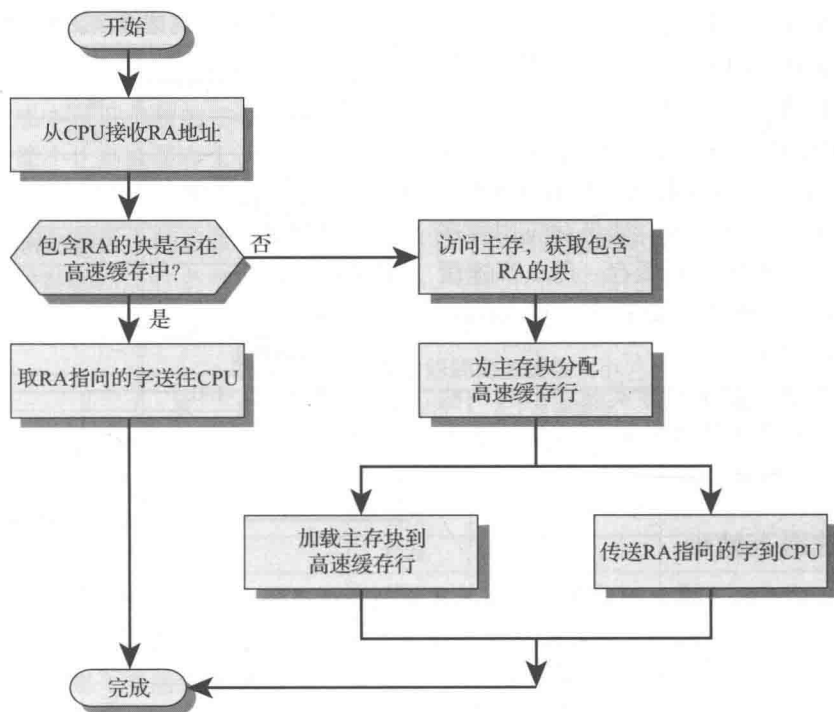


图 4-5 高速缓存读操作

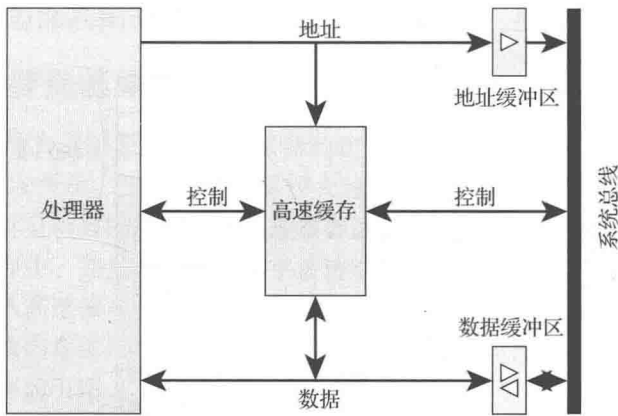


图 4-6 高速缓存典型结构

存的系统总线。当发生高速缓存命中时，数据和地址缓冲区被禁用，通信只在处理器和高速缓存之间进行，无系统总线通信。当发生高速缓存缺失时，所需地址被加载到系统总线，数据通过数据缓冲区返回给高速缓存和处理器。在其他结构中，对所有数据线、地址线和控制线来说，高速缓存在物理上位于处理器和主存之间。对这种情况，出现高速缓存缺失时，所需字首先被读入高速缓存，然后从高速缓存传递给处理器。

附录 4A 讨论了与高速缓存使用相关的性能参数。

4.3 高速缓存设计要素

本节介绍高速缓存设计参数，并报告一些典型结果。我们偶尔会提到高速缓存在高性能计算（HPC）中的使用。HPC 涉及超级计算机及其软件，尤其是包含大量数据、向量和矩阵计算，以及使用并行算法的科学应用。HPC 的高速缓存设计与其他硬件平台和应用的 高速缓存设计有着很大区别。实际上，许多研究者发现 HPC 应用程序在使用高速缓存的计算机体系结构上执行效果不佳 [BAIL93]。还有一些研究者逐渐证明，如果对应用程序软件进行调整以利用高速缓存，那么高速缓存层次结构可以用于提高性能 [WANG99, PRES01][⊖]。

尽管有大量的高速缓存实现，但有一些基本设计元素可以对高速缓存架构进行分类和区分。表 4-2 列出了关键元素。

表 4-2 高速缓存设计要素

高速缓存地址	写策略
逻辑地址	通写
物理地址	回写
高速缓存大小	行大小
映射函数	高速缓存数量
直接	一层或两层
全相联	混合或分离
组相联	
替换算法	
最近最少使用 (LRU)	
先进先出 (FIFO)	
最少使用 (LFU)	
随机	

4.3.1 高速缓存地址

几乎所有的非嵌入式处理器和许多嵌入式处理器都支持虚拟内存，第 8 章将讨论这个概念。从本质上来看，虚拟内存是一种允许程序从逻辑角度来寻址存储器的工具，它不从物理上考虑主存的可用容量。当使用虚拟内存时，机器指令的地址字段包含了虚拟地址。读写主

⊖ 有关 HPC 的一般讨论参见 [DOWD98]。

存时，存储器管理单元（MMU）硬件把每个虚拟地址转换为主存中的物理地址。

使用虚拟地址时，系统设计人员会选择把高速缓放在处理器与 MMU 之间，或是 MMU 与主存之间（图 4-7）。逻辑高速缓存，也称为虚拟高速缓存，用虚拟地址保存数据。处理器直接访问高速缓存，不用经过 MMU。物理高速缓存用主存物理地址保存数据。

逻辑高速缓存的一个明显优势是访问它的速度快于访问物理高速缓存，因为它可以在 MMU 执行地址转换之前进行响应。缺点在于，大多数虚拟内存系统为每个应用程序提供相同的虚拟内存地址空间。也就是说，每一个应用程序看到的虚拟内存都是从地址 0 开始的。因此，同一个虚拟地址在两个不同的应用程序中会指向两个不同的物理地址。所以，每个应用程序上下文交换时必须完全刷新高速缓存存储器，或是每个高速缓存行增加额外的位来指明这个地址指向的是哪个虚拟地址空间。

逻辑和物理高速缓存是个复杂的问题，本书不予讨论。关于这个问题更多更深入的讨论参见 [CEKL97] 和 [JACO08]。

4.3.2 高速缓存大小

表 4-2 中的第二项高速缓存大小已经讨论过了。我们既希望高速缓存足够小，以便其总体平均每位成本接近单独的主存，又希望它足够大，以便其整体平均访问时间接近于单独的高速缓存。最小化高速缓存大小还有其他几个动机。高速缓存越大，用于寻址该缓存的门就越多。其结果就是，较大的高速缓存往往比较小的要慢一些，即使使用集成电路技术相同，放置在芯片和电路板上的位置也相同。可用的芯片和电路板面积也限制了高速缓存大小。由于高速缓存的性能对工作负载的特性非常敏感，因此，不可能达到单个“最佳”高速缓存大小。表 4-3 列出了现在和过去一些处理器的高速缓存大小。

表 4-3 一些处理器的高速缓存大小

处理器	类 型	发布年份	L1 高速缓存 ^①	L2 高速缓存	L3 高速缓存
IBM 360/85	大型机	1968	16 ~ 32 kB	—	—
PDP-11/70	小型计算机	1975	1 kB	—	—
VAX 11/780	小型计算机	1978	16 kB	—	—
IBM 3033	大型机	1978	64 kB	—	—
IBM 3090	大型机	1985	128 ~ 256 kB	—	—
Intel 80486	PC	1989	8 kB	—	—
Pentium	PC	1993	8 kB/8 kB	256 ~ 512 kB	—
PowerPC 601	PC	1993	32 kB	—	—

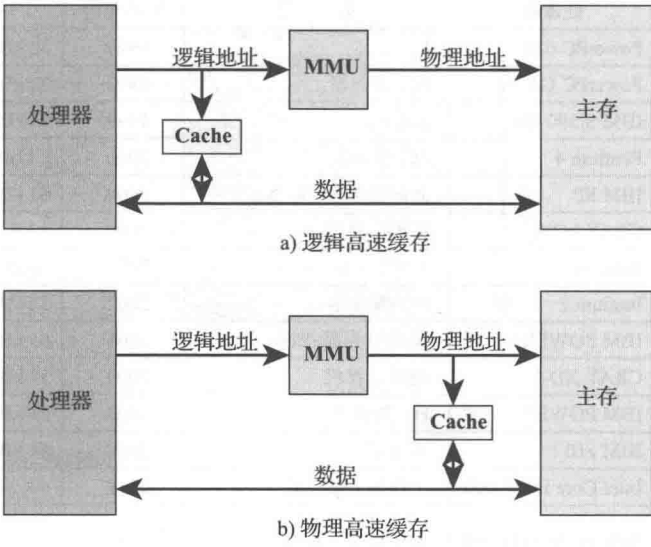


图 4-7 逻辑和物理高速缓存

(续)

处理器	类 型	发布年份	L1 高速缓存 ^①	L2 高速缓存	L3 高速缓存
PowerPC 620	PC	1996	32 kB/32 kB	—	—
PowerPC G4	PC/ 服务器	1999	32 kB/32 kB	256 kB ~ 1 MB	2 MB
IBM S/390 G6	大型机	1999	256 kB	8 MB	—
Pentium 4	PC/ 服务器	2000	8 kB/8 kB	256 kB	—
IBM SP	高端服务器 / 超级计算机	2000	64 kB/32 kB	8 MB	—
CRAY MTA ^②	超级计算机	2000	8 kB	2 MB	—
Itanium	PC/ 服务器	2001	16 kB/16 kB	96 kB	4 MB
Itanium 2	PC/ 服务器	2002	32 kB	256 kB	6 MB
IBM POWER5	高端服务器	2003	64 kB	1.9 MB	36 MB
CRAY XD-1	超级计算机	2004	64 kB/64 kB	1 MB	—
IBM POWER6	PC/ 服务器	2007	64 kB/64 kB	4 MB	32 MB
IBM z10	大型机	2008	64 kB/128 kB	3 MB	24~48 MB
Intel Core i7 EE 990	工作站 / 服务器	2011	6 × 32 kB/32 kB	1.5 MB	12 MB
IBM zEnterprise 196	大型机 / 服务器	2011	24 × 64 kB/128 kB	24 × 1.5 MB	24 MB L3 192 MB L4

①用斜杆分隔的两个值指的是指令和数据高速缓存。

②两个高速缓存都是指令的，没有数据高速缓存。

4.3.3 映射函数

由于高速缓存行比主存块少，需要用算法把主存块映射到高速缓存行。此外，还需要有方法来确定一个主存块当前要占用哪个高速缓存行。映射函数的选择决定了高速缓存的组织方式，使用的技术有三种：直接映射、全相联映射和组相联映射。下面将依次对它们进行讨论，对每一种技术，先介绍其一般结构，再给出一个具体例子。

例 4.2 对这三种技术，例子都包含下列条件：

- 高速缓存可以保存 64 kB。
- 数据以块为单位在主存和高速缓存之间传递，每块大小为 4 字节。这就是说，按每行 4 字节组织，高速缓存共有 $16\text{ K}=2^{14}$ 行。
- 主存包含 16 MB，每字节用一个 24 位 ($2^{24}=16\text{ M}$) 地址直接寻址。因此，为了映射，可以把主存看成 4 字节一个数据块，一共包含 4 M 个块。

直接映射 直接映射是最简单的技术，把每个主存块映射到唯一可能的高速缓存行。映射关系表示为

$$i=j \bmod m$$

其中：

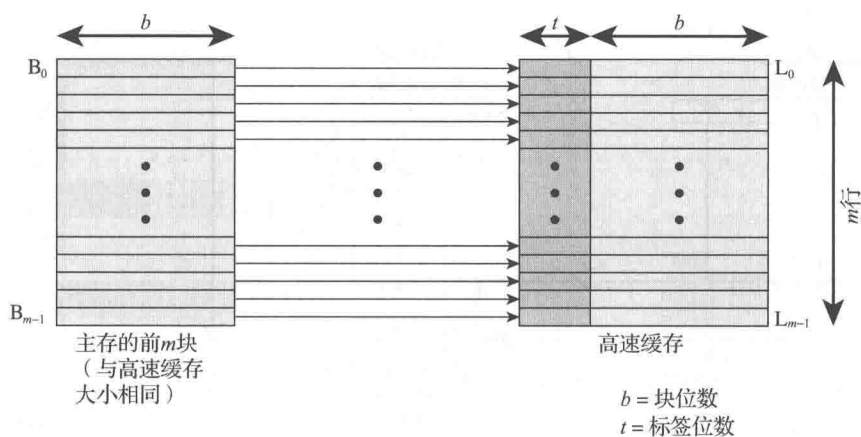
i = 高速缓存行号

j = 主存块号

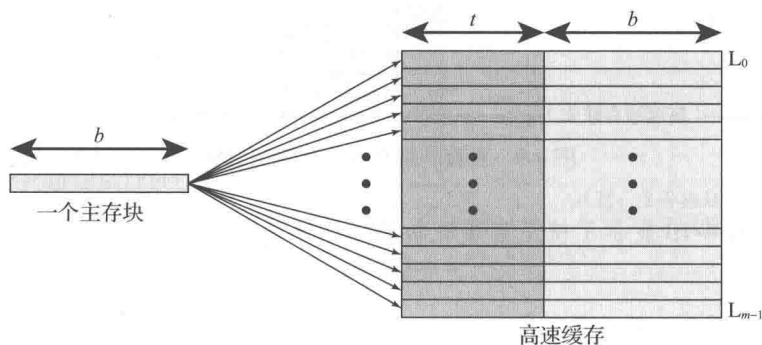
m = 高速缓存行总数

图 4-8a 展示了主存前 m 个数据块的映射。每个主存块被映射到高速缓存中唯一的一行。主存中下一组 m 块以同样的方式映射到高速缓存，即 B_m 主存块映射到 L_0 高速缓存行， B_{m+1}

主存块映射到 L_1 行，以此类推。



a) 直接映射



b) 全相联映射

图 4-8 从主存映射到高速缓存：直接映射和全相联映射

使用主存地址可以很容易地实现映射函数。图 4-9 展示了一般机制。对高速缓存访问来说，每个主存地址都可以被视为由三个字段构成。最低 w 位指明一个主存块内唯一的字或字节；对大多数当代机器来说，这个地址是字节级的。地址中的其余 s 位指定 2^s 个主存块中的一块。高速缓存逻辑把这 s 位解释为一个 $(s-r)$ 位的标签字段（最高位部分）和一个 r 位的行字段。后者指明 $m=2^r$ 个高速缓存行中的一行。总之：

- 地址长度 = $(s+w)$ 位
- 可寻址单元个数 = 2^{s+w} 个字或字节
- 块大小 = 行大小 = 2^w 个字或字节
- 主存中块的个数 = $\frac{2^{s+w}}{2^w} = 2^s$
- 高速缓存中的行数 = $m=2^r$
- 高速缓存大小 = 2^{r+w} 个字或字节
- 标签大小 = $(s-r)$ 位

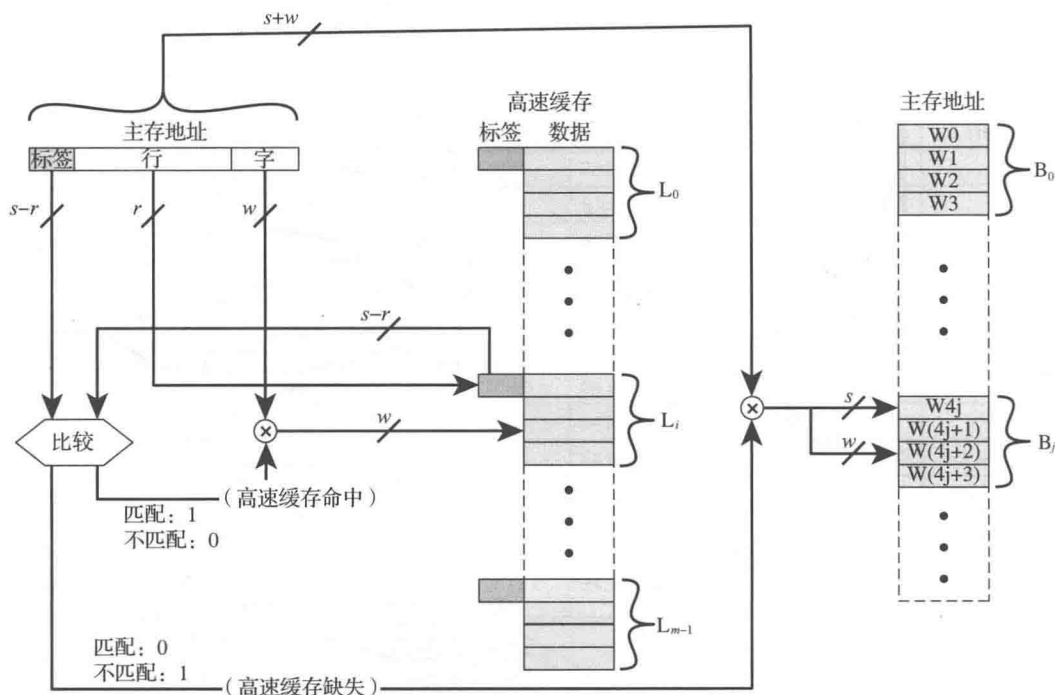


图 4-9 直接映射高速缓存组织结构

例 4.2a 图 4-10 显示了使用直接映射的示例系统[⊖]。本例中， $m=16$ $K=2^{14}$ ， $i=j \bmod 2^{14}$ 。映射如下：

高速缓存行	块在内存中的起始地址
0	000000, 010000, ..., FF0000
1	000004, 010004, ..., FF0004
⋮	⋮
$2^{14}-1$	00FFFC, 01FFFC, ..., FFFFFC

注意映射到同一行上的两个块有着不同的标签号。因此，起始地址为 000000, 010000, …, FF0000 的块其标签号分别为 00, 01, …, FF。

回顾一下图 4-5，读操作的工作方式是这样的。高速缓存系统用 24 位地址表示。16 位行号用于在高速缓存中检索，以访问特定行。如果 8 位标签与该行当前存储的标签匹配，那么 2 位的字号就用于在这个行的 4 字节中选择一个。否则，就用 22 位的标签+行字段从主存中取数据块。用于获取的实际地址是 22 位的标签+行字段，再加上两个 0 位，这样获取的 4 字节就会起始于块边界。

⊖ 本图与后续各图中，内存值用十六进制符号表示。有关数字系统的基础复习（十进制、二进制、十六进制）参见第 9 章。

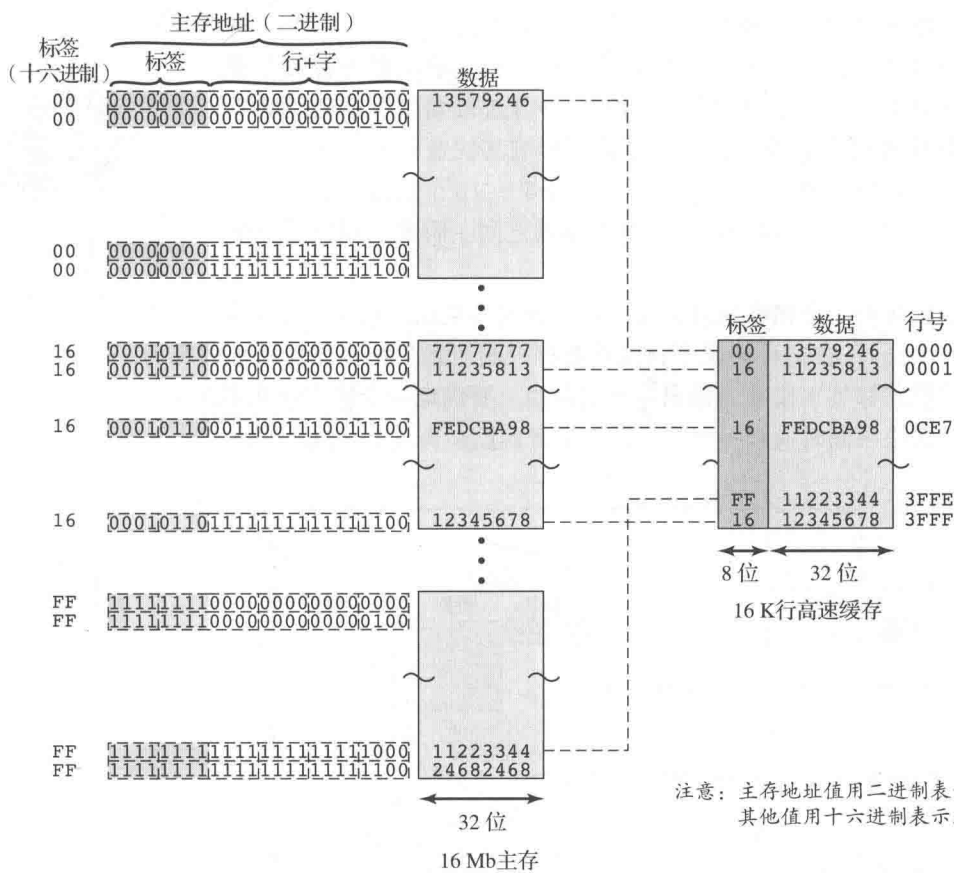


图 4-10 直接映射示例

这种映射的效果是把主存块分配到高速缓存行，如下所示：

高速缓存行	分配的主存块
0	$0, m, 2m, \dots, 2^s - m$
1	$1, m+1, 2m+1, \dots, 2^s - m + 1$
$\vdots$	$\vdots$
$m-1$	$m-1, 2m-1, 3m-1, \dots, 2^s - 1$

因此，用地址的一部分作为行号，就能提供每个主存块到高速缓存的唯一映射。当一个块被实际读入分配给它的行时，就必须对数据进行标记，以便将其与其他适合该行的块进行区分。最高有效的 $s-r$ 位就是做这个用的。

直接映射技术实现起来简单且便宜。它的主要缺点是对任何给定块都只有固定的高速缓存位置。因此，如果一个程序反复从两个不同块中引用的字正好映射到同一个行，那么这些块就会不断地被交换进高速缓存，从而降低命中率（这种现象被称为抖动）。

降低缺失代价的一种方法是保存被丢弃的内容，以防它再次被需要。由于被丢弃的数据已经被获取过，所以再次使用它的成本很小。使用受害者缓存能进行这种回收。受害者缓存最初作为一种方法被提出，是为了减少直接映射高速缓存的冲突缺失，而又不影响其快速访问时间。受害者缓存是全相联高速缓存，其大小一般为4~16个高速缓存行，位于直接映射的L1高速缓存和下一级存储器之间。附录F探讨了这个概念。



可选的受害者缓存模拟器

全相联映射 全相联映射克服了直接映射的缺点，它允许每个主存块加载到任何高速缓存行(图4-8b)。在这种情况下，高速缓存控制逻辑只把存储器地址解释为一个标签字段和一个字字段。标签字段唯一标识一个主存块。要确定一个主存块在不在高速缓存中，高速缓存控制逻辑必须同时检查每个行的标签是否匹配。图4-11展示这种逻辑。

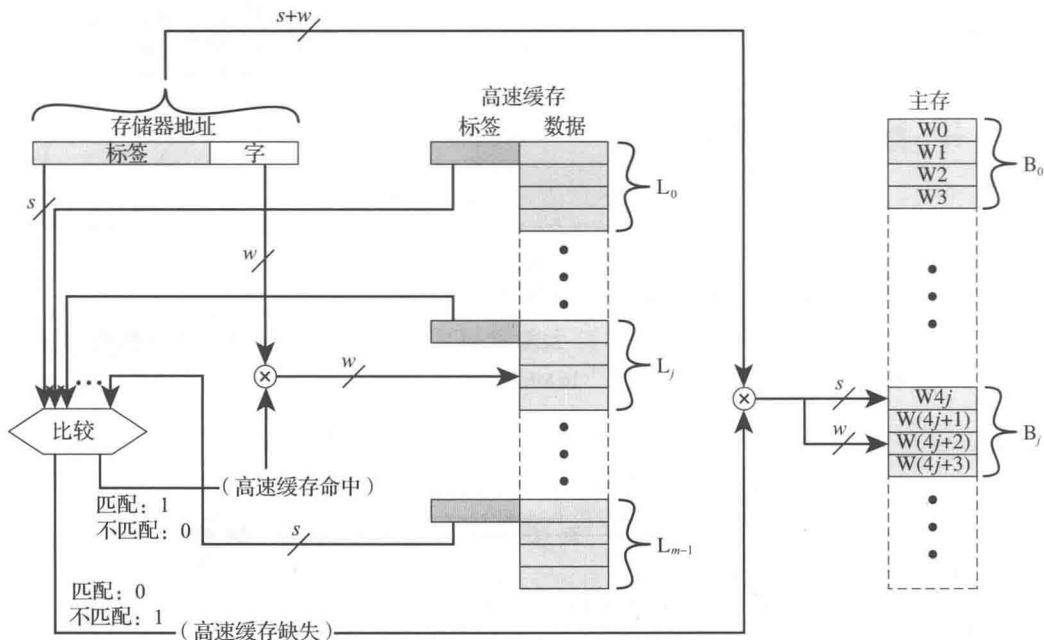


图4-11 全相联高速缓存组织结构

例 4.2b 图4-12展示了使用全相联映射的例子。主存地址由22位的标签和2位的字节号构成。22位标签必须与每个高速缓存行的32位数据块一起保存在高速缓存中。注意地址最左边(最高有效)的22位形成了标签。所以，24位十六进制地址16339C的22位标签为058CE7。用二进制表示更容易看出来：

存储器地址	0001	0110	0011	0011	1001	1100	(二进制)
	1	6	3	3	9	C	(十六进制)
标签(最左边的22位)	00	0101	1000	1100	1110	0111	(二进制)
	0	5	8	C	E	7	(十六进制)

注意地址中没有字段对应于行号，因此，高速缓存中行数不由地址格式决定。总之：

- 地址长度 = $(s+w)$ 位

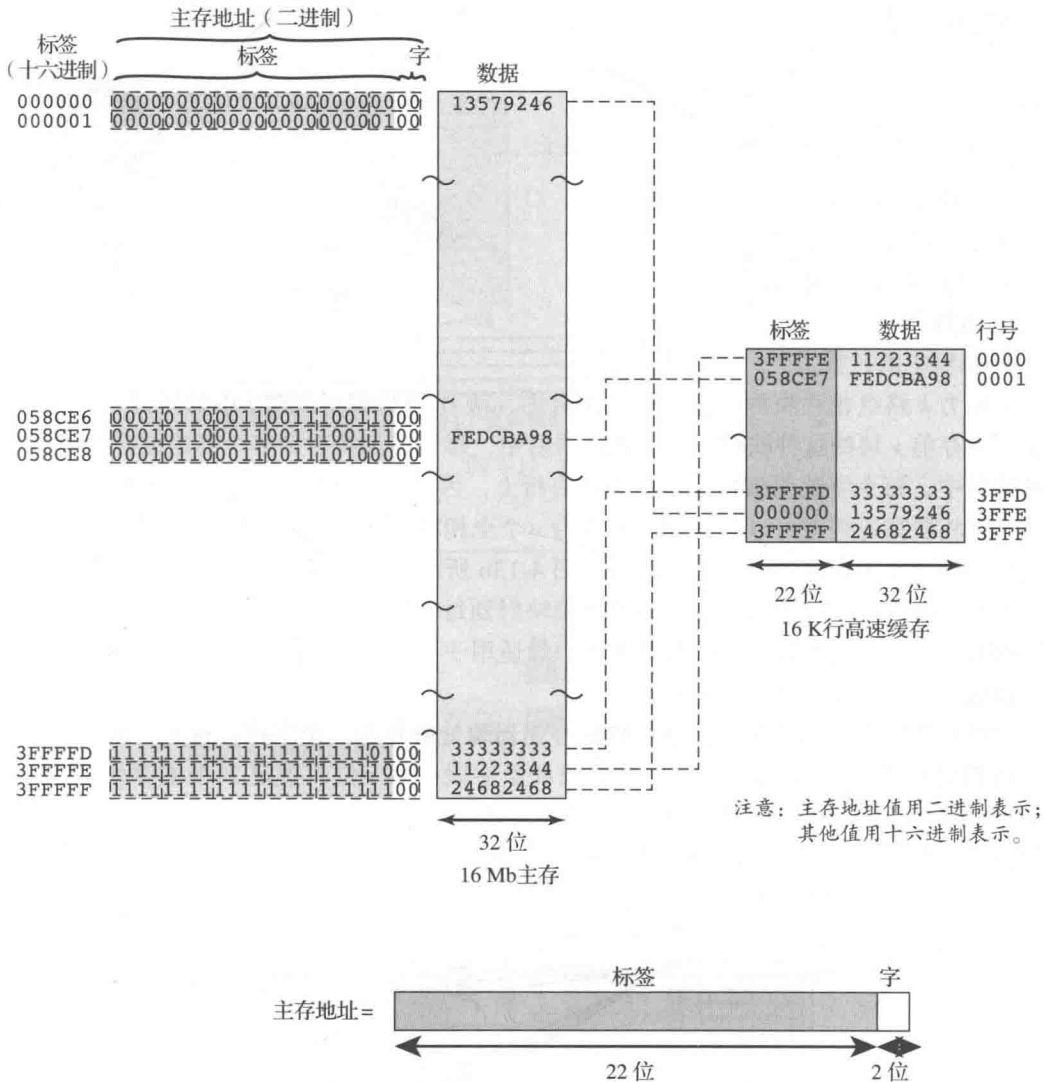


图 4-12 全相联映射示例

- 可寻址单元个数 = 2^{s+w} 个字或字节
- 块大小 = 行大小 = 2^w 个字或字节
- 主存中块的个数 = $\frac{2^{s+w}}{2^w} = 2^s$
- 高速缓存中的行数 = 未定
- 标签大小 = s 位

当新块被读入高速缓存时，使用全相联映射可以灵活地决定替换哪个块。替换算法旨在最大化命中率，本节稍后讨论。全相联映射的主要缺点是需要复杂的电路来并行检查全部高速缓存行的标签。

组相联映射 组相联映射是一种折中方案，它既体现了直接映射和全相联映射的优点，又减少了它们的不足。

在这种情况下, 高速缓存由一些组构成, 每一组都包含了若干行。关系如下:

$$m = v \times k$$

$$i = j \text{ modulo } v$$

其中:

i = 高速缓存组号

j = 主存块号

m = 高速缓存中的行数

v = 组数

k = 每组中的行数

这称为 k 路组相联映射。在组相联映射中, 块 B_j 可以映射到组 j 中的任何行。图 4-13a 显示了主存前 v 块的这种映射。在全相联映射中, 每个字都映射到多个高速缓存行中。在组相联映射中, 每个字映射到特定组内的所有行上, 因此, 主存块 B_0 映射到组 0, 以此类推。所以, 组相联高速缓存可以在物理上实现为 v 个全相联高速缓存。同理, 还可以把组相联高速缓存实现为 k 个直接映射高速缓存, 如图 4-13b 所示。每个直接映射高速缓存都被称为一路, 由 v 行组成。主存的第一组 v 行被直接映射到每一路的 v 行上; 主存的下一组 v 行也进行同样的映射, 以此类推。直接映射实现一般适用于小相联度 (k 值小), 而全相联映射实现则一般适用于较高相联度 [JACO08]。

对组相联映射, 高速缓存控制逻辑把存储器地址解释为三个字段: 标签、组和字。 d 位组字段指定 $v=2^d$ 个组中的一个。标签字段和组字段一共 s 位, 指定 2^s 个主存块中的一个。图 4-14 展示了这种高速缓存控制逻辑。使用全相联映射, 存储器地址中的标签非常大, 且必须与高速缓存中每一行的标签比较。使用 k 路组相联映射, 存储器地址中的标签要小得多, 且只需与一个组中的 k 个标签比较。

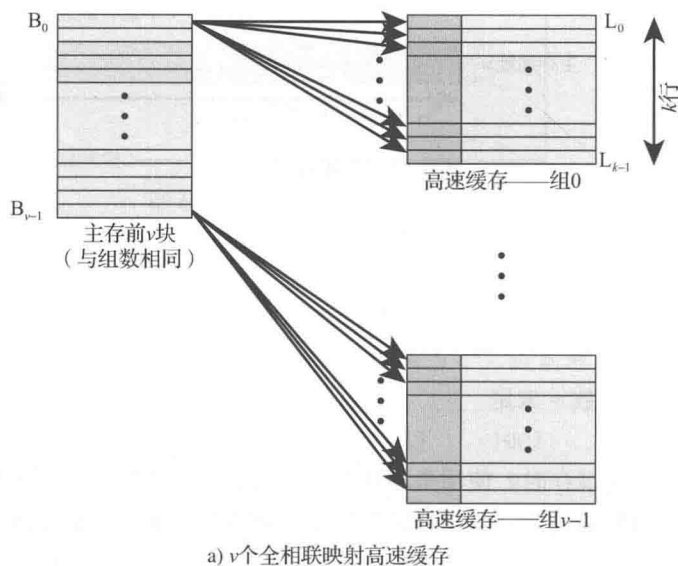


图 4-13 从主存到高速缓存的映射: k 路组相联



高速缓存时间分析模拟器

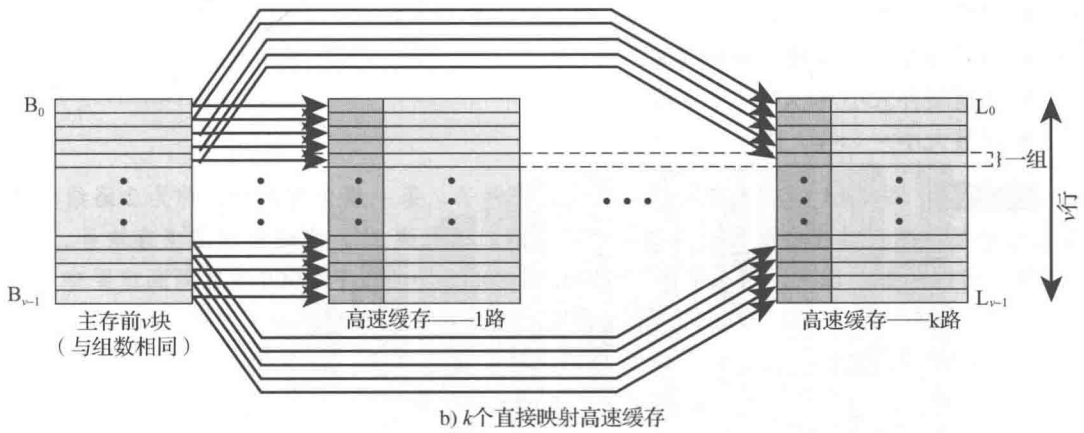
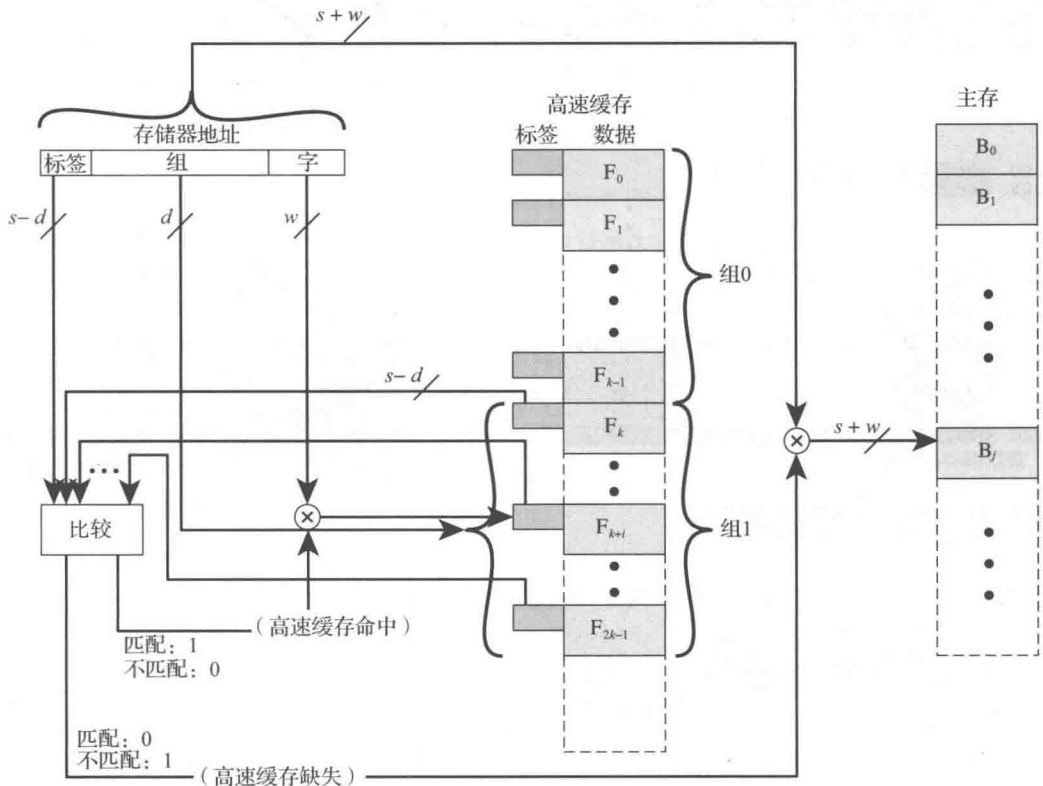


图 4-13 (续)



总之：

- 地址长度 = $(s+w)$ 位
- 可寻址单元个数 = 2^{s+w} 个字或字节
- 块大小 = 行大小 = 2^w 个字或字节
- 主存中块的个数 = $\frac{2^{s+w}}{2^w} = 2^s$
- 组中行数 = k

- 组数 $=v=2^d$
- 高速缓存中的行数 $=m=kv=k \times 2^d$
- 高速缓存大小 $=k \times 2^{d+w}$ 个字或字节
- 标签大小 $= (s-d)$ 位

例 4.2c 图 4-15 给出了使用组相联映射的例子，每一组含有两行，称为 2 路组相联。13 位组号标识高速缓存中由两行组成的唯一一组。通过模 2^{13} ，它还给出了主存块号。这决定了块到行的映射。因此，主存中块 000000，008000，……，FF8000 映射到高速缓存 0 组。这些块中的任何一个都可以加载到这个组内两行的任一行。注意，如果有两个块映射到高速缓存同一组，那么它们的标签号是不一样的。对读操作而言，13 位组号用于确定要检查的是哪个两行组。组内两行都要检查是否与访问地址的标签号匹配。

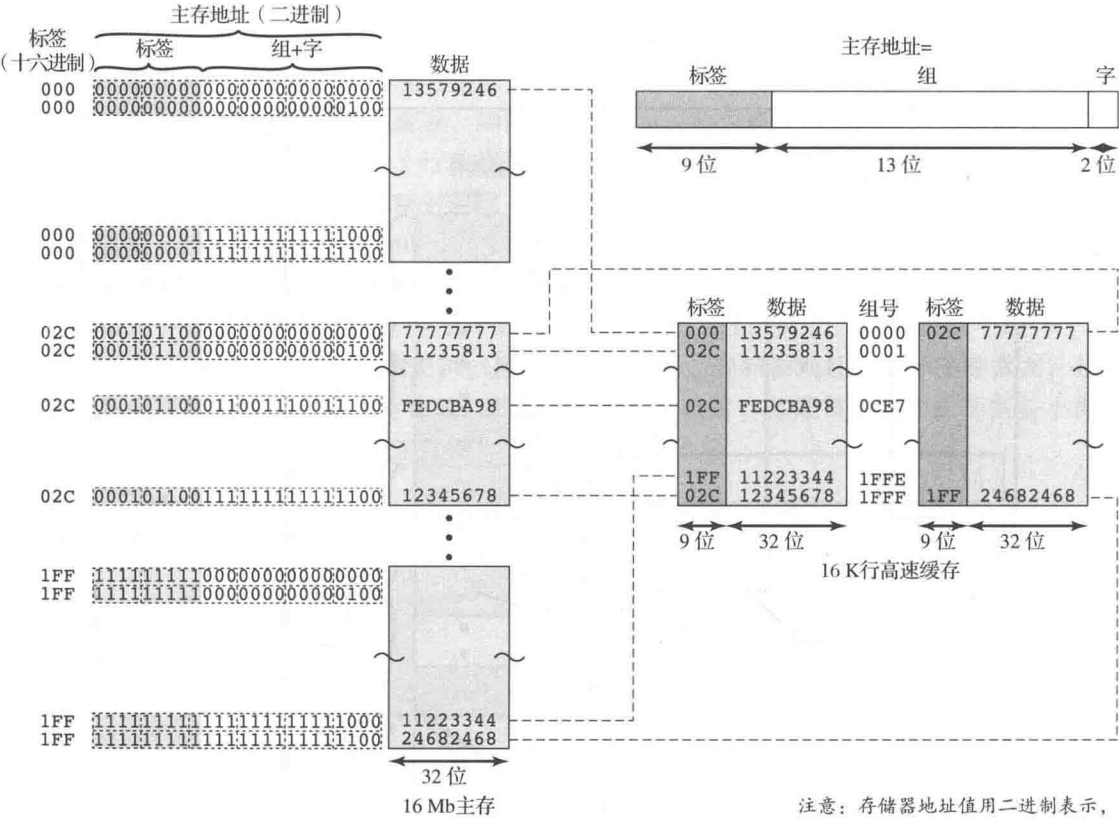


图 4-15 2 路组相联映射示例

在 $v=m$ ， $k=1$ 的极端条件下，组相联技术简化为直接映射，当 $v=1$ ， $k=m$ 时，则简化为全相联映射。使用每组两行 ($v=m/2$ ， $k=2$) 是最常见的组相联结构。与直接映射相比，它显著提高了命中率。4 路组相联 ($v=m/4$ ， $k=4$) 用相对较小的额外成本取得了适度的更多改进 [MAYB84, HILL89]。每组中行数的进一步增加则收效甚微。

图 4-16 显示了一项模拟研究的结果，即组相联高速缓存的性能与高速缓存大小的函数关系 [GENU04]。高速缓存大小低于 64 kB 时，直接映射和 2 路组相联映射的性能差异非常显著。还要注意，4 kB 大小时 2 路和 4 路的差别远小于高速缓存大小从 4 kB 到 8 kB 时两者

之间的差异。高速缓存的复杂度与关联度的增加成正比, 这种情况下, 把高速缓存大小增加到 8 kB 甚至 16 kB 是不合理的。最后要注意的一点是, 超过 32 kB 后, 增加高速缓存大小不会带来性能的显著提高。

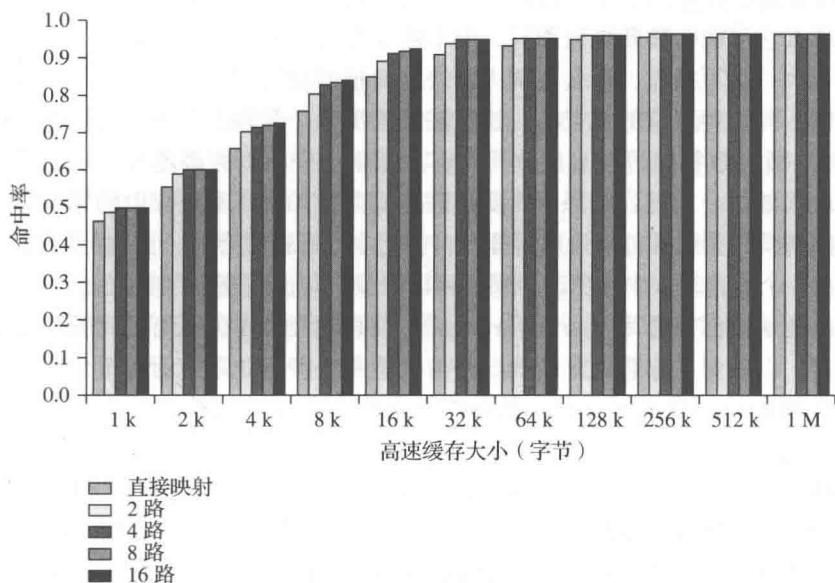


图 4-16 不同的相联度与高速缓存大小的关系

图 4-16 的结果是以 GCC 编译器的执行为基础的。不同的应用程序可能产生不同的结果。比如, [CANT01] 使用许多 CPU2000 SPEC 基准测试程序报告高速缓存性能结果。[CANT01] 命中率与高速缓存大小的比较结果与图 4-16 的模式相同, 但具体数值略有不同。



高速缓存模拟器
多任务高速缓存模拟器

4.3.4 替换算法

当高速缓存已满, 而又有新块要调入时, 就必须替换其中已存在的块。对直接映射而言, 任何特定块都只有一个可能的行与之对应, 没有其他选择。对全相联和组相联技术而言, 就需要替换算法。为了达到高速, 算法必须在硬件上实现。许多算法都被尝试过, 我们只讨论最常见的 4 种。最有效的大概是最近最少使用 (LRU) 算法: 替换组内在高速缓存中最久没有被引用过的块。在两路组相联中, 这个算法很容易实现。每个行有一个 USE 位。当某行被引用后, 其 USE 位置 1, 而同组中另一行的 USE 位清 0。当一个块被读入该组时, 使用的是 USE 位为 0 的行。由于我们假设最近使用的存储器地址更有可能被引用, 所以 LRU 算法应该具有最佳命中率。对全相联高速缓存, 实现 LRU 算法也相对容易。高速缓存机制为缓存中全部行维护一个单独的索引列表。当一行被引用时, 它被移动到列表的前端。替换时, 使用的是列表后端的行。由于其实现简单, LRU 是最流行的替换算法。

另一种方法是先进先出 (FIFO) 算法: 替换组内存在高速缓存中最久的块。FIFO 很容易实现为循环或循环缓冲技术。还有一种可能的方法是最不常用 (LFU) 算法: 替换组内经历引用最少的块。LFU 可以通过在每行关联一个计数器来实现。不基于用法的技术 (即不是 LRU、LFU、FIFO 或某些变体) 是从候选行中随机选择一行。仿真研究表明, 随机替换的

性能仅略低于基于用法的算法 [SMIT82]。

4.3.5 写策略

当驻留在高速缓存中的一个块被替换时，要考虑两种情况。如果高速缓存中的旧块没有被更新过，那么它可能会被新块覆盖写，而不需要先写出旧块。如果高速缓存的该行中某个字被执行了至少一次写操作，那么在调入新块前要把旧块写到主存块来更新主存。根据性能与成本的权衡，有各种可能的写策略。这里需要解决两个问题。首先，可能有多个设备访问主存。例如，一个 I/O 模块可以直接读写主存。如果一个字只在高速缓存中被更改，那么相应的内存字就无效了。此外，如果 I/O 设备更改了主存，则高速缓存中的字就无效了。当多个处理器连接到同一总线且每个处理器都有自己的本地高速缓存时，出现的问题就更加复杂。如果一个字在一个高速缓存中被更改，那么可以想象，其他高速缓存中的这个字就会无效。

最简单的技术被称为**通写** (write through)。使用这个技术，所有的写操作都会施加于主存和高速缓存，以确保主存总是有效的。任何其他的处理器-高速缓存模块都可以监视到主存的传送，以维护自己高速缓存的一致性。这个技术主要的缺点是它会产生大量的主存流量，可能会形成一个瓶颈。另一种技术是**回写** (write back)，它能最小化主存写操作。对回写而言，更新仅在高速缓存中执行。发生更新时，与行关联的脏 (dirty) 位或使用 (use) 位置 1。之后，当一个块被替换时，当且仅当 dirty 位为 1 时，该块被写回主存。回写的问题是部分主存是无效的，因此，I/O 模块只能通过高速缓存访问。这造成了复杂的电路和潜在的瓶颈。经验表明，写入的内存引用百分比大约为 15% [SMIT82]。然而，对 HPC 应用来说，这个百分比可能接近 33% (向量-向量乘法)，且最高可达 50% (矩阵转置)。

例 4.3 考虑一个行大小为 32 字节的高速缓存，主存传输一个 4 字节的字需要 30 ns。任何行在被交换出高速缓存之前至少要被写一次，那么对一个回写高速缓存来说，为了比通写缓存更高效，数据行在被交换出去之前平均被写次数是多少？

对回写情况而言，每个脏数据行在交换时间回写一次，花费 $8 \times 30 = 240$ ns。对通写情况而言，数据行的每次更新都需要把一个字写到主存，花费 30 ns。因此，如果至少写一次数据行在交换出去之前被写次数超过 8 次，那么回写就更高效。

在总线组织中，有多个设备 (通常是处理器) 带有高速缓存且共享主存，这会产生新的问题。如果一个高速缓存中的数据被修改了，那么不仅会使得主存中相应的字无效，也会使得其他高速缓存中同一个字无效 (如果其他高速缓存恰好有相同的字)。即使使用通写策略，其他高速缓存也可能包含无效数据。防止这个问题的系统被称为维护高速缓存一致性。高速缓存一致性包含如下可能的方法：

- **通写的总线监视** 每个高速缓存控制器监视地址线，以检测其他总线主机对主存的写操作。如果另一个主机向共享内存写入的位置在高速缓存中也有驻留，那么高速缓存控制器就使得该缓存项无效。这个策略依赖于所有高速缓存控制器对通写策略的使用。
- **硬件透明** 用额外的硬件确保所有通过高速缓存对主存的更新都反映到全部高速缓存中。因此，如果一个处理器在自己的高速缓存中修改了一个字，这个更新就会被写入主存。此外，任何其他高速缓存中相同的字也会进行类似的更新。
- **不可高速缓存存储器** 只有一部分主存被多个处理器共享，这被指定为不可高速缓

存的。在这样的系统中，所有对共享存储器的访问都是高速缓存缺失，因为共享存储器不会复制到高速缓存中。不可高速缓存存储器可以用片选逻辑和高位地址来识别。

高速缓存一致性是研究的活跃领域，这个主题将在第五部分进行更深入的探讨。

4.3.6 数据行大小

另一个设计要素是行大小。当一个数据块被取出并放入高速缓存时，不仅仅是所需字，一些相邻字也被取出。当块大小从很小增加到很大时，由于局部性原理，命中率会先升高，这表示被引用字附近的数据在不远的将来很可能也会被引用。当块大小增加时，更多有用的数据被放入高速缓存。但是块变得更大时，命中率却开始下降，因为使用新获取到信息的概率变得小于重用被替换信息的概率。有两个具体的影响：

- 更大的块减少了高速缓存能容纳的块数。由于每个获取到的块都会覆盖原来的高速缓存内容，因此，块数少会导致获取到的数据很快就被覆盖。
- 当数据块变大时，每个额外的字距离被请求的字会更远，因此，在不远的将来不太可能被需要。

块大小与命中率之间的关系是复杂的，它取决于特定程序的局部特性，找不到确定的最优值。8 ~ 64 字节的大小似乎是最接近最优的 [SMIT87, PRZY88, PRZY90, HAND98]。对 HPC 系统来说，64 字节和 128 字节高速缓存行大小是最常用的。

4.3.7 高速缓存的数量

在最初引入高速缓存的时候，典型系统只包含一个高速缓存。最近，使用多个高速缓存越来越常见。此设计问题涉及两个方面：高速缓存的层数以及混合和分离缓存的使用。

多层高速缓存 随着逻辑密度的增加，让高速缓存与处理器位于同一芯片上成为可能：片上高速缓存。相比于通过外部总线访问高速缓存，片上缓存减少了处理器外部总线的活动，因此也就加快了执行速度降低了执行时间，提高了系统整体性能。当被请求的指令或数据在片上高速缓存中被发现时，总线访问被取消。由于处理器内部的路径较短，与总线长度相比，片上高速缓存访问甚至比零等待状态总线周期还要快得多。此外，在这个访问期间，总线可以用来支持其他传输。

包含片上高速缓存会留下一个问题，即是否还需要片外或外部高速缓存。通常情况下，答案是肯定的，大多数现代设计都包括了片上和外部高速缓存。这种组织结构最简单的就是两级高速缓存：内部 1 级（L1）和指定为 2 级（L2）的外部高速缓存。包含 L2 高速缓存的理由是：如果没有 L2 高速缓存，当处理器对一个不存在于 L1 缓存的主存位置发出访问请求时，处理器就必须通过总线访问 DRAM 或 ROM 存储器。由于总线速度通常较慢，存储器访问时间通常较长，这就会导致性能不佳。另一方面，如果使用了 L2 SRAM（静态 RAM）高速缓存，则可以经常快速检索到缺失的信息。如果 SRAM 的速度足以匹配总线速度，那么数据就能以零等待状态事务进行访问，这是最快的总线传输类型。

多级高速缓存的现代缓存设计有两个特性值得注意。其一，对片外 L2 高速缓存来说，许多设计没有使用系统总线作为 L2 高速缓存和处理器之间的传输路径，而是使用了单独的数据路径，这样就可以减轻系统总线的负担。其二，随着处理器组件的不断缩小，现在许多处理器在处理器芯片上集成了 L2 高速缓存，以提高性能。

使用 L2 高速缓存可能节省的成本取决于 L1 和 L2 高速缓存的命中率。一些研究表明,一般而言,使用二级高速缓存确实可以提高性能(比如,参见 [AZIM92], [NOVI93], [HAND98])。但是,使用多级高速缓存会使所有有关缓存的设计问题复杂化,包括大小、替换算法、写策略;讨论参见 [HAND98] 和 [PEIR99]。

图 4-17 显示了一个两级高速缓存性能随缓存大小变化的仿真研究结果 [GENU04]。该图假设两个高速缓存的行大小相同,并显示了总命中率。也就是说,如果所需数据在 L1 或 L2 高速缓存中找到,那么计为一次命中。图中展示了相对于 L1 的大小, L2 对总命中率的影 响。L2 对高速缓存命中总数的影响较小,直到其大小至少达到 L1 缓存大小的两倍。请注意 8 kB L1 高速缓存坡度最陡峭的部分对应的是 16 kB 的 L2 高速缓存。同样,对 16 kB L1 高速缓存来说,曲线最陡峭的部分对应的是 32 kB 的 L2 缓存大小。在此之前, L2 高速缓存对总体性能的影响(如果有)非常小。让 L2 高速缓存容量大于 L1 缓存来影响性能是有道理的。如果 L2 缓存与 L1 缓存的行大小和容量都一样,则其内容或多或少会是 L1 缓存内容的镜像。

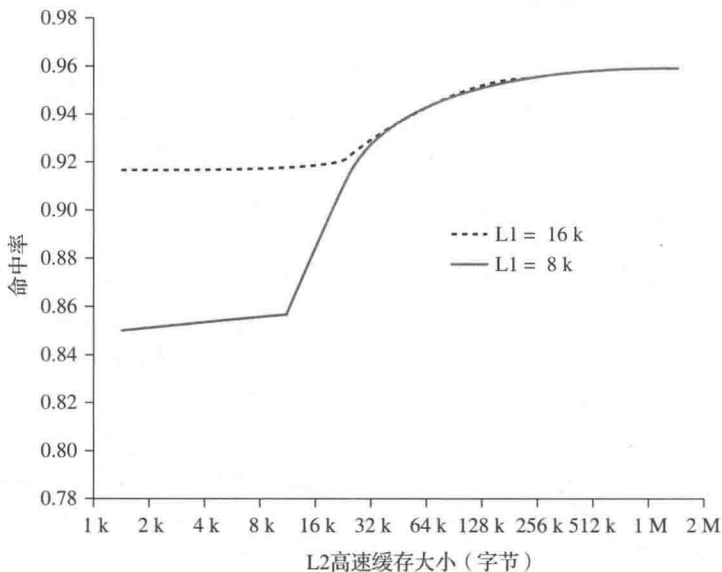


图 4-17 8 kB 和 16 kB L1 下的总命中率 (L1 和 L2)

随着芯片上可用高速缓存区域增大,大多数现代微处理器都把 L2 缓存移到了芯片上,并添加了 L3 缓存。开始的时候, L3 高速缓存要通过外部总线访问。最近,大多数微处理器都添加了片上 L3 缓存。在这两种情况下,增加第三级缓存似乎都能提高性能(如,参见 [GHAI98])。此外,像 IBM 大型机 zEnterprise 系统那样的大型系统现在还合并了三级片上高速缓存,并在多个芯片之间共享第四级缓存 [CURR11]。

混合式与分离式高速缓存 当片上高速缓存首次出现时,许多设计都由一个缓存组成,用于存储被引用的数据和指令。最近变得常见的是把高速缓存分成两个:一个专门存指令,一个专门存数据。这两个高速缓存处于同一级别,通常为两个 L1 缓存。当处理器试图从主存取指令时,它首先查询指令 L1 缓存;当处理器试图从主存取数据时,它首先查询数据 L1 缓存。

混合式高速缓存有两个潜在的优势：

- 对给定高速缓存大小来说，混合式高速缓存的命中率高于分离式，因为它能自动平衡取指令和取数据的负载。也就是说，如果执行模式包含的取指操作比取数操作要多，那么高速缓存就会被指令填满；如果执行模式包含的取数操作相对较多，则会出现相反的情况。
- 只需设计和实现一个高速缓存。

目前的趋势是在 L1 上分离高速缓存，在更高层次上使用混合式缓存，尤其是在强调指令并行执行和预取被预测未来指令的超标量计算机上。分离式高速缓存设计的主要优点是它消除了指令预取 / 译码单元和执行单元对高速缓存的争用。这一点对依赖于指令流水执行的任何设计都是重要的。通常，处理器会提前取指令，并用要执行的指令来填充缓冲区或流水线。假设现在我们有一个混合式指令 / 数据高速缓存。当执行单元执行加载和存储数据的存储器访问时，该请求被提交到这个混合式高速缓存。如果与此同时，指令预取部件也向高速缓存发出一条指令的读请求，这个请求就会被暂时阻塞，以便高速缓存先为执行单元服务，使其完成当前执行的指令。这种高速缓存争用会干扰指令流水线的使用效率，从而降低性能。分离式高速缓存结构可以克服这个困难。

4.4 Pentium 4 高速缓存结构

高速缓存结构的演进在 Intel 微处理器的演变过程中得到了清晰的体现（表 4-4）。80386 没有包含片上高速缓存。80486 包含了一个 8 kB 的片上高速缓存，其行大小为 16 字节，4 路组相联。所有的 Pentium 处理器都包含两个片上 L1 高速缓存，一个是数据的，一个是指令的。对 Pentium 4 来说，L1 数据高速缓存为 16 kB，行大小为 64 字节，4 路组相联。稍后介绍 Pentium 4 指令高速缓存。Pentium II 也含有一个 L2 高速缓存，它为两个 L1 缓存提供信息。L2 缓存是 8 路组相联的，大小为 512 kB，行大小为 128 字节。Pentium III 增加了一个 L3 高速缓存，并在高端版本的 Pentium 4 中成为片上缓存。

表 4-4 Intel 高速缓存的演变

问 题	解决方案	出现该特性的第一个处理器
外部存储器比系统总线慢	使用更快的存储技术添加外部高速缓存	386
处理器速度的提高使得外部总线成为高速缓存访问的瓶颈	把外部高速缓存移到片上，操作速度与处理器相同	486
由于芯片空间有限，内部高速缓存相当小	使用比主存更快的技术添加外部 L2 高速缓存	486
当指令预取部件和执行单元同时请求对高速缓存的访问，则发生争用。在这种情况下，预取部件被阻塞，发生执行单元的数据访问	创建各自独立的数据和指令高速缓存	Pentium
处理器速度的提高使得外部总线成为访问 L2 高速缓存的瓶颈	创建单独的后端总线（BSB），其运行速度快于主（前端）总线。BSB 专门用于 L2 高速缓存	Pentium Pro
	把 L2 高速缓存移到处理器芯片上	Pentium II
某些应用程序处理大量数据库，必须能快速访问大量数据。片上高速缓存太小	添加外部 L3 高速缓存	Pentium III
	把 L3 高速缓存移到片上	Pentium 4

图 4-18 提供了 Pentium 4 组成的简化视图，突出显示了三个高速缓存的位置。处理器核心由 4 个部分组成：

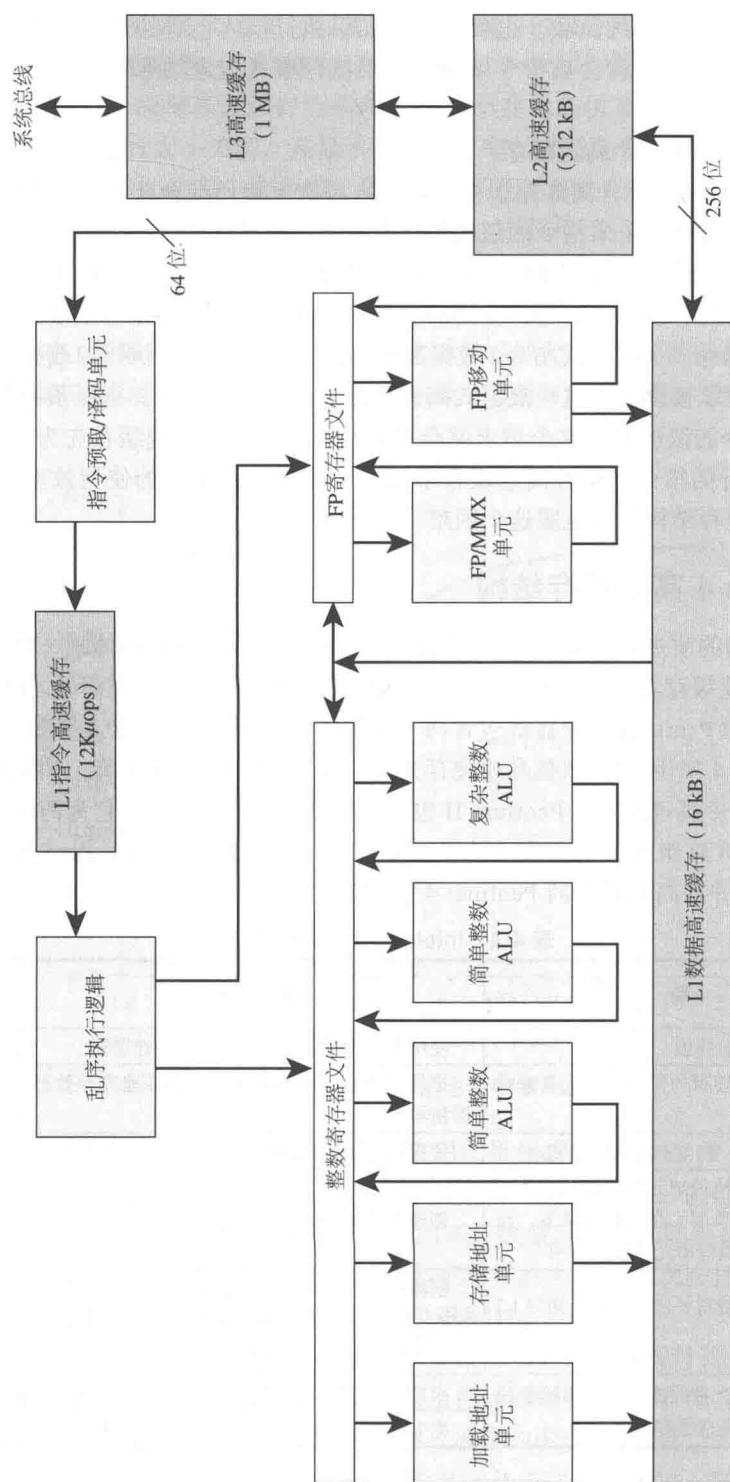


图 4-18 Pentium 4 框图

- **预取 / 译码单元** 按序从 L2 高速缓存中取出程序指令，把它们译码为一系列微操作，并把结果存放在 L1 指令高速缓存中。
- **乱序执行逻辑** 根据数据依赖性与资源可用性调度微操作的执行，因此，微操作执行的调度顺序可能与它们从指令流中取出的顺序不一致。在时间允许的情况下，这个单元对将来可能需要的微操作进行推测执行调度。
- **执行单元** 这些单元执行微操作，从 L1 数据高速缓存中取得所需数据，并将结果临时存放在寄存器中。
- **存储器子系统** 这个单元包含了 L2、L3 高速缓存以及系统总线，用于在 L1 和 L2 高速缓存出现缓存缺失时访问主存，以及访问系统 I/O 资源。

与之前所有的 Pentium 模型以及大多数其他处理器所使用的组成不一样的是，Pentium 4 的指令高速缓存位于指令译码逻辑和执行核心之间。这种设计决策背后的原因如下：正如第 16 章更全面的讨论所展示的，Pentium 过程把 Pentium 机器指令译码或翻译为简单的类 RISC 指令，称为微操作。使用简单的，定长微操作可以利用超标量流水技术和调度技术来提高性能。但是，Pentium 机器指令译码很烦琐，它们的字节数可变，且有许多不同的选项。结果是，如果这种译码是独立于调度和流水逻辑，就能提高性能。我们将在第 16 章回到这个主题。

数据高速缓存使用了回写策略：仅当数据从高速缓存中移除且已经被更新过，那么它们会被写回主存。Pentium 4 处理器可以进行动态配置来支持通写高速缓存。

L1 数据高速缓存受控于一个控制寄存器中的两位，它们被标识为 CD（禁止高速缓存）位和 NW（非通写）位（表 4-5）。还有两条 Pentium 4 指令可以用于控制数据高速缓存：INVD 是内部高速缓存失效（刷新），并向外部高速缓存（如果有）发出无效信号。WBINVD 写回内部缓存并使其无效，然后写回外部缓存并使其无效。

L2 和 L3 高速缓存都是 8 路组相联结构，行大小为 128 字节。

表 4-5 Pentium 4 高速缓存操作模式

控制位		操作模式		
CD	NW	高速缓存填充	通写	无效
0	0	允许	允许	允许
1	0	禁止	允许	允许
1	1	禁止	禁止	禁止

注：CD=0，NW=1 是无效组合。

4.5 关键词、复习题和练习题

关键词

access time: 访问时间

associative mapping: 全相联映射

cache hit: 高速缓存命中

direct mapping: 直接映射

High-Performance Computing (HPC): 高性能计算

hit: 命中

hit ratio: 命中率

instruction cache: 指令高速缓存

L1 cache: L1 高速缓存

L2 cache: L2 高速缓存

L3 cache: L3 高速缓存

line: 行	sequential access: 顺序访问
locality: 局部性	set-associative mapping: 组相联映射
cache line: 高速缓存行	cache set: 高速缓存组
cache memory: 高速缓存存储器	data cache: 数据高速缓存
cache miss 高速缓存缺失	direct access: 直接访问
logical cache: 逻辑高速缓存	spatial locality: 空间局部性
memory hierarchy: 存储器层次	split cache: 分离式高速缓存
miss: 缺失	tag: 标签
multilevel cache: 多级高速缓存	temporal locality: 时间局部性
physical address: 物理地址	unified cache: 混合式高速缓存
physical cache: 物理高速缓存	virtual address: 虚拟地址
random access: 随机访问	virtual cache: 虚拟高速缓存
replacement algorithm: 替换算法	write back: 回写
secondary memory: 辅助存储器	write through: 通写

复习题

- 4.1 顺序访问、直接访问和随机访问之间有何差异?
- 4.2 访问时间、存储器成本和容量之间的一般关系是什么?
- 4.3 使用多级存储器与局部性原则之间有何关联?
- 4.4 直接映射、全相联映射和组相联映射之间有何差异?
- 4.5 对直接映射高速缓存来说,主存地址被看作由三个字段构成。列出并定义这三个字段。
- 4.6 对全相联映射高速缓存来说,主存地址被看作由两个字段构成。列出并定义这两个字段。
- 4.7 对组相联映射高速缓存来说,主存地址被看作由三个字段构成。列出并定义这三个字段。
- 4.8 空间局部性和时间局部性的不同是什么?
- 4.9 一般而言,利用空间局部性和时间局部性的策略是什么?

练习题

- 4.1 某组相联高速缓存含有 64 行,分为 4 个组。主存有 4 K 个数据块,每块为 128 字。请给出主存地址。
- 4.2 某 2 路组相联高速缓存行大小为 16 字节,总容量为 8 kB。64 MB 的主存为字节寻址。请给出主存地址。
- 4.3 对十六进制的主存地址 111111, 666666, BBBB, 请用十六进制的形式给出如下信息:
 - a. 用图 4-10 的格式,给出直接映射高速缓存的标签字段、行字段和字字段的值。
 - b. 用图 4-12 的格式,给出全相联映射高速缓存的标签字段和字字段的值。
 - c. 用图 4-15 的格式,给出 2 路组相联映射高速缓存的标签字段、组字段和字字段的值。
- 4.4 列出如下各值:
 - a. 对图 4-10 中的直接映射高速缓存示例:地址长度、可寻址单元数、块大小、主存块数、高速缓存行数、标签的大小。
 - b. 对图 4-12 中的全相联映射高速缓存示例:地址长度、可寻址单元数、块大小、主存块数、高速缓存行数、标签的大小。
 - c. 对图 4-15 中的 2 路组相联映射高速缓存示例:地址长度、可寻址单元数、块大小、主存块数、

组内行数、组数、高速缓存行数、标签的大小。

- 4.5 考虑一个 32 位微处理器，其片上高速缓存为 16 kB，4 路组相联。假设该高速缓存的行大小为 4 个 32 位字。画出它的方框图，要求展示其组织结构，以及如何使用不同的地址字段来确定高速缓存命中或缺失。内存地址为 ABCDE8F8 的字映射到这个高速缓存的什么位置？
- 4.6 给定外部高速缓存参数如下：4 路组相联，行大小为两个 16 位字，能够容纳来自主存的总共 4 K 个 32 位字，与发出 24 位地址的 16 位处理器一起工作。使用所有相关信息设计这个缓存的结构，展示它是如何解释处理器地址的。
- 4.7 Intel 80486 包含一个片上分离式高速缓存。该缓存的大小为 8 kB，4 路组相联结构，块长度为 4 个 32 位字。该缓存被组织成 128 组。每行有一个“行有效位”和 B0、B1、B2 三个“LRU”位。当高速缓存缺失时，80486 通过总线内存突发读从内存读取一个 16 字节的行。绘制该缓存的简图，展示如何解释地址的不同字段。
- 4.8 考虑一个机器，其字节寻址的主存容量为 2^{16} 字节，块大小为 8 字节。假设该机器使用的直接映射高速缓存有 32 行。
- 16 位的地址如何被分成标签字段、行号字段和字节号字段？
 - 下列地址中的字节将被保存到哪个行上？

0001	0001	0001	1011
1100	0011	0011	0100
1101	0000	0001	1101
1010	1010	1010	1010
 - 假设地址为 0001 1010 0001 1010 的字节保存在这个缓存中，那么和它一起存储的其他字节的地址是什么？
 - 总共有多少内存字节能保存到这个高速缓存中？
 - 为什么标签也保存在高速缓存中？
- 4.9 对其片上高速缓存来说，Intel 80486 使用了被称为伪最近最少使用的替换算法。128 组 4 行（标记为 L0、L1、L2、L3）内的每一行都有 3 位 B0、B1、B2 与其关联。替换算法的工作原理如下：当必须替换一行时，高速缓存将首先确定最近一次使用的是 L0 和 L1，还是 L2 和 L3。然后再确定这一对块中哪一个最近被使用过，并将其标记为要替换。图 4-19 说明了其中的逻辑。

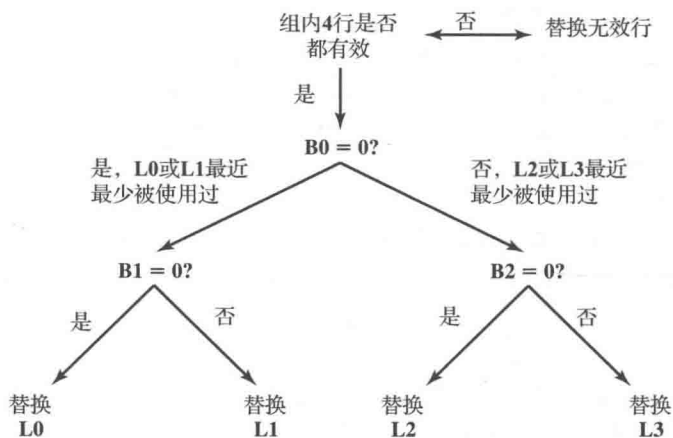


图 4-19 Intel 80486 片上高速缓存替换策略

- 说明 B0、B1、B2 位是如何设置的，并用文字解释在图 4-19 所表示的替换算法中它们是如何被

使用的。

- b. 证明 80486 的算法近似于真正的 LRU 算法。提示：考虑最近使用顺序为 L0, L2, L3, L1 的情况。
 - c. 证明真正的 LRU 算法每组需要 6 位。
- 4.10 一个组相联高速缓存的块大小为 4 个 16 位字，组大小为 2。该缓存能容纳总共 4096 个字。可缓存的主存大小为 64 K × 32 位。设计缓存结构，并展示如何解释处理器地址。
- 4.11 考虑一个存储系统使用 32 位地址对字节进行寻址，其高速缓存的行大小为 64 字节。
- a. 假设该高速缓存为直接映射，地址中的标签字段为 20 位。给出地址格式并确定如下参数：可寻址单元的个数、主存块数、高速缓存行数、标签大小。
 - b. 假设该高速缓存为全相联映射。给出地址格式并确定如下参数：可寻址单元的个数、主存块数、高速缓存行数、标签大小。
 - c. 假设该高速缓存为 4 路组相联映射，地址中的标签字段为 9 位。给出地址格式并确定如下参数：可寻址单元的个数、主存块数、组内行数、高速缓存组数、高速缓存行数、标签大小。
- 4.12 考虑具有如下特性的计算机：主存容量为 1 MB，字大小为 1 字节，块大小为 16 字节，高速缓存容量为 64 kB。
- a. 对直接映射高速缓存，给出主存地址为 F0010、01234 和 CABBE 的单元对应的标签、高速缓存行地址以及字偏移量。
 - b. 对直接映射高速缓存，给出任意两个具有不同标签的主存地址，它们被映射到同一个缓存行上。
 - c. 对全相联映射高速缓存，给出主存地址为 F0010 和 CABBE 的单元对应的标签和字偏移量。
 - d. 对 2 路组相联映射高速缓存，给出主存地址为 F0010 和 CABBE 的单元对应的标签、高速缓存组号和字偏移量。
- 4.13 描述一种简单的技术在 4 路组相联高速缓存中实现 LRU 替换算法。
- 4.14 再次考虑例 4.3。如果主存的块传输能力变为第一个字的访问时间为 30 ns，其后每个字的访问时间都为 5 ns，那么答案会如何变化？
- 4.15 考虑如下代码：
- ```
for (i = 0; i < 20; i++)
 for (j = 0; j < 10; j++)
 a[i] = a[i]*j
```
- a. 给出上述代码空间局部性的一个例子。
  - b. 给出上述代码时间局部性的一个例子。
- 4.16 将附录 4A 中的式 (4.2) 和式 (4.3) 推广到  $N$  级存储器层次结构。
- 4.17 某计算机的主存容量为 32 K 个 16 位字，其 4 K 字的高速缓存被分为若干组，每组 4 行，每行 64 个字。假设初始时高速缓存为空，处理器按顺序从位置 0, 1, 2, …, 4351 取字。然后再重复这个取字序列 9 次。高速缓存的速度比主存快 10 倍。请估算使用高速缓存带来的改进。假设块替换算法为 LRU。
- 4.18 考虑一个 4 行的高速缓存，每行 16 字节。主存按 16 字节分块，也就是说，块 0 包含地址 0 ~ 15 的字节，以此类推。现在考虑一个程序按如下地址顺序访问存储器：
- 一次：63 ~ 70
- 10 次循环：15 ~ 32；80 ~ 95
- a. 假设高速缓存按直接映射组织。主存块 0、4 等被分配到缓存行 1；主存块 1、5 等被分配到缓存行 2，以此类推。计算命中率。
  - b. 假设高速缓存按 2 路组相联映射组织，共 2 组，每组 2 行。偶数块被分配到组 0，奇数块被分配到组 1。利用最近最少使用替换模式，计算这个 2 路组相联高速缓存的命中率。
- 4.19 考虑一个有如下参数的存储系统：

$$\begin{array}{ll} T_c = 100 \text{ ns} & C_c = 10^{-4} \text{ 美元/位} \\ T_m = 1200 \text{ ns} & C_m = 10^{-5} \text{ 美元/位} \end{array}$$

- a. 1 MB 主存的成本是多少?
- b. 使用高速缓存技术的 1 MB 主存的成本是多少?
- c. 如果有效访问时间比高速缓存访问时间多 10%, 那么命中率  $H$  是多少?
- 4.20 a. 考虑一个 L1 高速缓存, 其访问时间为 1 ns, 命中率  $H=0.95$ 。若我们可以改变它的设计 (缓存大小、缓存组成) 使得  $H$  提高到 0.97, 而访问时间增加到 1.5 ns。为了提高性能, 必须满足哪些条件才能实现这个改变?
- b. 请解释为什么这个结果具有直观意义。
- 4.21 考虑一个单级高速缓存, 其访问时间为 2.5 ns, 行大小为 64 字节, 命中率  $H=0.95$ 。主存的块传输能力为第一个字 (4 字节) 的访问时间为 50 ns, 后续各字的访问时间为 5 ns。
- a. 当高速缓存缺失时, 访问时间是多少? 假设高速缓存一直等到该行从主存取出, 然后重新执行命中。
- b. 若行大小增加到 128 字节, 会使  $H$  提高到 0.97, 这会降低平均内存访问时间吗?
- 4.22 某计算机有高速缓存、主存和用于虚拟存储器的硬盘。如果一个被引用的字在高速缓存中, 那么访问它需要 20 ns。如果这个字在主存中而不在高速缓存中, 那么需要 60 ns 将其加载到高速缓存, 然后再次启动对它的访问。如果这个字不在主存中, 则需要 12 ms 把它从硬盘中取出, 之后用 60 ns 复制到高速缓存, 再重启对它的访问。高速缓存命中率为 0.9, 主存命中率为 0.6。那么在这个系统上访问一个被引用字所需的平均时间是多少?
- 4.23 考虑一个高速缓存, 其行大小为 64 字节。假设缓存中平均 30% 的行是被写过的。一个字有 8 字节。
- a. 若缺失率为 3% (命中率为 0.97)。针对通写和回写策略, 以字节数 / 指令为单位计算主存流量。主存一次向高速缓存读入一行。但是, 对于回写, 可以把单个字从高速缓存写入主存。
- b. 若缺失率为 5%, 重做上题。
- c. 若缺失率为 7%, 重做上题。
- d. 从上述结果能得出什么结论?
- 4.24 在 Motorola 68020 微处理器中, 访问高速缓存需要 2 个时钟周期。没有等待状态插入的情况下, 处理器通过总线从主存访问数据需要 3 个时钟周期; 数据传递给处理器与传递给高速缓存是并行的。
- a. 给定命中率为 0.9 和时钟频率为 16.67 MHz, 计算存储周期的平均时长。
- b. 假设每个存储周期的每个时钟都插入两个等待状态, 重做上题。从结果能得出什么结论?
- 4.25 假设某处理器的存储器周期时间为 300 ns, 指令处理速率为 1 MIPS。平均而言, 每条指令需要一个总线存储周期取指令, 一个总线存储周期取涉及的操作数。
- a. 计算处理器对总线的利用率。
- b. 假设处理器配有一个指令高速缓存, 且相关命中率为 0.5。请确定对总线使用的影响。
- 4.26 单级高速缓存系统的读操作的性能可以用如下公式表示:
- $$T_a = T_c + (1 - H)T_m$$
- 其中,  $T_a$  是平均访问时间,  $T_c$  是高速缓存访问时间,  $T_m$  是内存访问时间 (内存到处理器寄存器),  $H$  是命中率。为了简化, 我们假设被请求的字同时加载到高速缓存和处理器寄存器中。这与式 (4.2) 的形式相同。
- a. 定义  $T_b$  为在高速缓存与主存之间传输一行所需时间,  $W$  = 写引用的比例。使用通写策略, 修改前面的公式, 以考虑写和读。
- b. 定义  $W_b$  为高速缓存中一行被修改的概率。为回写策略提供表示  $T_a$  的公式。
- 4.27 某系统有两级高速缓存, 定义  $T_{c1}$  = 第一级高速缓存访问时间;  $T_{c2}$  = 第二级高速缓存访问时间;

$T_m$  = 内存访问时间;  $H_1$  = 第一级高速缓存命中率;  $H_2$  = 第一级 / 第二级高速缓存整体命中率。为读操作提供表示  $T_a$  的公式。

- 4.28 假设某高速缓存读缺失性能特点如下: 1 个时钟周期向主存发送地址, 4 个时钟周期从主存访问一个 32 位字并将其传送到处理器和高速缓存。
- a. 若高速缓存行大小为 1 个字, 则缺失代价是多少 (即读缺失事件中需要的额外读取时间)?
  - b. 若高速缓存行大小为 4 个字, 且执行非突发多传输, 则缺失代价是多少?
  - c. 若高速缓存行大小为 4 个字, 且执行传输时每个时钟周期传输一个字, 则缺失代价是多少?
- 4.29 对上述练习题中的高速缓存设计, 假设行大小从 1 个字增加到 4 个字会导致读缺失率从 3.2% 降到 1.1%。对非突发传输和突发传输两种情况而言, 在所有的读操作中, 两种不同行大小的平均缺失代价是多少?

附录 4A 两级存储的性能特征

本章中引用一个高速缓存, 它充当了主存和处理器之间的缓冲区, 从而创建了一个两级的内部存储器。这个两级架构利用一种被称为局部性的特性, 提供了比单级存储器更好的性能。

主存 - 高速缓存机制是计算机体系结构的一部分, 由硬件实现, 通常对操作系统是不可见的。还有另外两个二级存储器方法的实例也利用了局部性, 并且至少是部分在操作系统中实现了: 虚拟存储器和磁盘缓存 (表 4-6)。虚拟存储器将在第 8 章讨论, 磁盘缓存超出了本书范围, 但在 [STAL15] 中进行了讨论。本附录将研究这三种方法所共有的二级存储的一些性能特性。

表 4-6 二级存储的特性

|           | 主存 - 高速缓存          | 虚拟存储器 (分页)                   | 磁盘缓存                         |
|-----------|--------------------|------------------------------|------------------------------|
| 典型的访问时间之比 | 5:1 (主存 : 高速缓存)    | 10 <sup>6</sup> :1 (主存 : 磁盘) | 10 <sup>6</sup> :1 (主存 : 磁盘) |
| 存储器管理系统   | 由特殊硬件实现            | 硬件和系统软件的组合                   | 系统软件                         |
| 典型块大小或页大小 | 4 ~ 128 字节 (高速缓存块) | 64 ~ 4096 字节 (虚拟存储器页)        | 64 ~ 4096 字节 (磁盘块或页)         |
| 处理器对二级的访问 | 直接访问               | 间接访问                         | 间接访问                         |

局部性

二级存储器性能优势的基础是被称为引用局部性的原理 [DENN68]。这个原理指出存储器引用倾向于簇聚。在较长时间段里, 正在使用的部分会变化, 但在较短时间段里, 处理器主要处理固定部分的存储器引用。

直观地说, 局部性原理是有意义的。考虑如下原因:

1) 程序执行是顺序的, 除了分支和调用指令, 但它们只构成所有程序指令的一小部分。因此, 大多数情况下, 下一条获取的指令会紧跟上一条获取的指令。

2) 长时间不间断的过程调用序列后跟相应的返回序列是很少见的。相反, 一个程序仍然会被限制在一个相当窄的过程调用深度的窗口。因此, 短时间的指令引用往往会局限于几个过程中。

3) 大多数迭代结构是由数量相对较少的指令重复多次构成的。因此在迭代过程中, 计算被限制在一小段连续的程序中。

4) 在很多程序中, 许多计算涉及对数据结构的处理, 比如数组或记录序列。在很多情况下, 对这些数据结构的连续引用会指向位置紧邻的数据项。

这些理由已经在许多研究中得到证实。参考第 1 点，各种研究分析了高级语言程序的行为。表 4-7 包含了以下研究的关键结果，测量了执行过程中各种语言的表现。Knuth[KNUT71] 最早对编程语言行为进行了研究，考察了一系列用作学生练习的 FORTRAN 程序。Tanenbaum[TANE78] 发表了从 300 多个用于操作系统程序的过程中收集到的测量值，这些过程用支持结构化编程（SAL）的语言编写。Patterson 和 Sequein [PATT82a] 分析了一组从编译器和程序中提取的测量值，这些程序用于排版、计算机辅助设计（CAD）、排序和文件比较。研究还包括了编程语言 C 和 Pascal。Huck[HUCK83] 分析了 4 个旨在表示通用科学计算混合的程序，包括快速傅里叶变换和微分方程系统的集成。这种语言和程序混合的结果有很好的 consistency，分支和调用指令只代表程序生命周期中执行语句的一小部分。因此，这些研究证实了断言 1。

表 4-7 高级语言操作的相对动态频率

| 语言负载研究 | [HUCK83]<br>Pascal 科学 | [KNUT71]<br>FORTRAN 学生 | [PATT82a] |      | [TANE78]<br>SAL 系统 |
|--------|-----------------------|------------------------|-----------|------|--------------------|
|        |                       |                        | Pascal 系统 | C 系统 |                    |
| 赋值     | 74                    | 67                     | 45        | 38   | 42                 |
| 循环     | 4                     | 3                      | 5         | 3    | 4                  |
| 调用     | 1                     | 3                      | 15        | 12   | 12                 |
| IF     | 20                    | 11                     | 29        | 43   | 36                 |
| GOTO   | 2                     | 9                      | —         | 3    | —                  |
| 其他     | —                     | 7                      | 6         | 1    | 6                  |

对于断言 2，[PATT85a] 报告的研究提供了证明。如图 4-20 所示，它展示了调用 - 返回行为。每次调用横线都会向下向右移动，每次返回横线都会向上向右移动。图中定义了一个深度等于 5 的窗口。只有在任意方向上净移动量为 6 的调用和返回序列才会使得窗口移动。可以看出，执行程序可以长时间保持在一个固定窗口内。对 C 和 Pascal 进行相同分析的研究表明，只要不到 1% 的调用或返回，深度为 8 的窗口就需要移动 [TAMI83]。

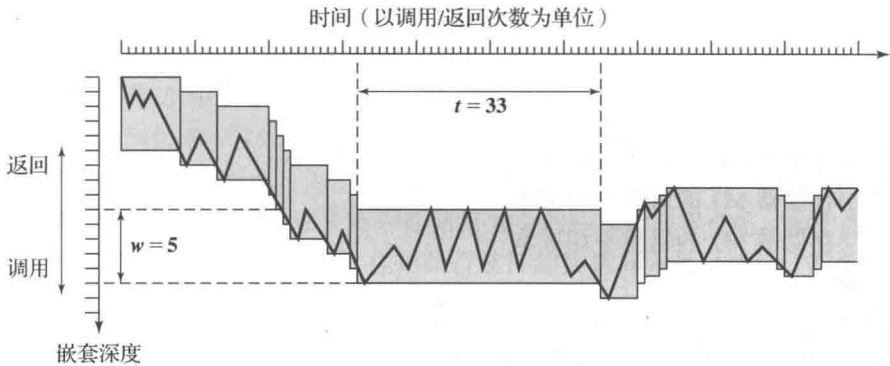


图 4-20 程序的调用 - 返回示例

文献对空间局部性和时间局部性进行了区分。**空间局部性**是指执行涉及的内存地址具有簇聚倾向。这反映了处理器按序访问指令的趋势。空间位置还反映了程序按序访问数据地址的趋势，比如当处理表格数据的时候。**时间局部性**是指处理器访问最近使用过的内存地址的倾向。举个例子，当执行迭代循环时，处理器重复执行一组相同的指令。

传统上，通过把最近使用的指令和数据值保存到高速缓存已经使用缓存层次结构来利用时间局部性。空间局部性则一般是通过使用更大的高速缓存块以及把预取机制（获取预期使



用的项)合并到缓存控制逻辑来利用。最近,已经有相当多的研究对这些技术进行了改进,以获得更好的性能,但基本策略仍然是一样的。

## 两级存储的操作

局部性可以用来形成两级存储。相对于下层存储器(M2),上层存储器(M1)更小、更快、更贵(每位)。M1暂时存放更大的M2的部分内容。当有内存引用时,将尝试访问M1中的项。如果成功,则进行快速访问。如果不成功,则把内存位置的数据块从M2复制到M1,然后通过M1进行访问。由于局部性的原因,当一个块被送入M1时,就应该有许多对该块内位置的访问,从而实现快速的整体服务。

要表示访问一项的平均时间,我们不仅要考虑两级存储的速度,还要考虑给定引用在M1中被找到的概率。由此

$$\begin{aligned} T_s &= H \times T_1 + (1-H) \times (T_1 + T_2) \\ &= T_1 + (1-H) \times T_2 \end{aligned} \quad (4.2)$$

其中:

$T_s$  = 平均(系统)访问时间

$T_1$  = M1的访问时间(如高速缓存、磁盘缓存)

$T_2$  = M2的访问时间(如主存、磁盘)

$H$  = 命中率(访问在M1中被找到的时间比例)

图4-2展示了平均访问时间与命中率的函数关系。可以看出,对高命中率来说,相较于M2,平均整体访问时间更接近于M1。

## 性能

让我们来看看与两级存储机制评估相关的一些参数。首先考虑成本:

$$C_s = \frac{C_1 S_1 + C_2 S_2}{S_1 + S_2} \quad (4.3)$$

其中:

$C_s$  = 两级存储组合的每位平均成本

$C_1$  = 上层存储器M1的每位平均成本

$C_2$  = 下层存储器M2的每位平均成本

$S_1$  = M1的大小

$S_2$  = M2的大小

我们希望  $C_s \approx C_2$ 。假设  $C_1 \gg C_2$ , 这要求  $S_1 < S_2$ 。图4-21展示了这种关系。

接下来,考虑访问时间。要向两级存储提供显著的性能提升,我们需要使得  $T_s$  近似等于  $T_1$  ( $T_s \approx T_1$ )。假设  $T_1$  比  $T_2$  小得多 ( $T_1 \ll T_2$ ), 命中率就需要接近于1。

所以,我们希望M1小一点来降低成本,大一点来提高命中率,从而提高性能。M1的大小能在一定程度上同时满足这两个需求吗?我们可以用一系列的子问题来回答它:

- 使  $T_s \approx T_1$  的命中率是多少?
- 为了保证所需命中率, M1 的大小应该是多少?
- 这个大小能满足成本要求吗?

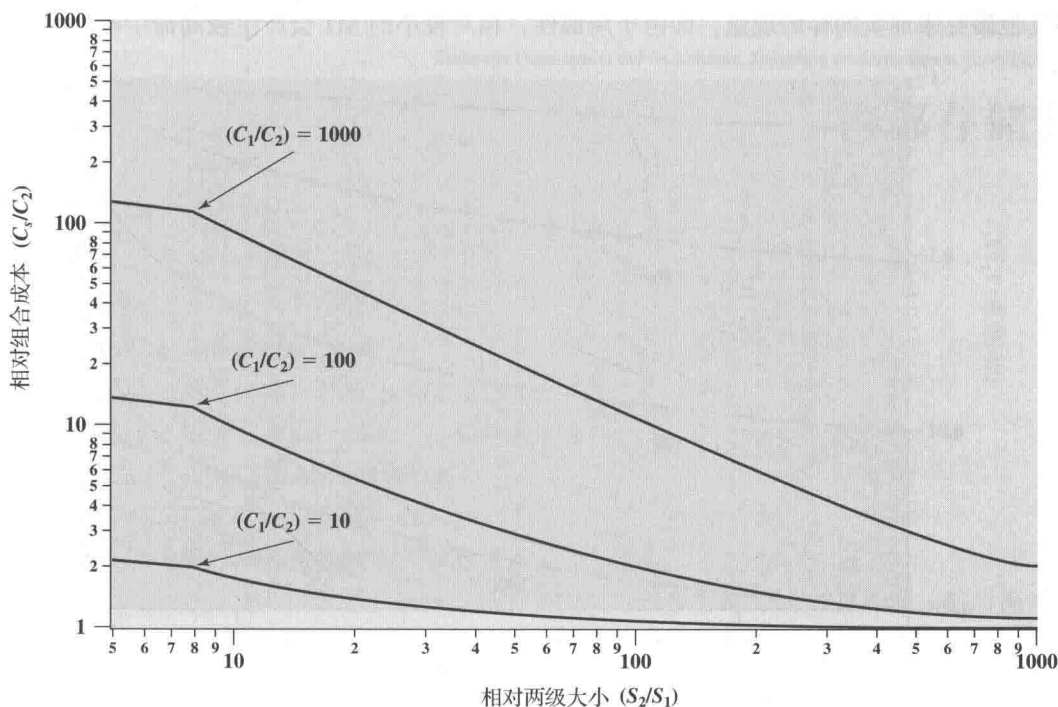


图 4-21 两级存储中平均存储器成本与相对存储器大小的关系

为了解决这个问题, 考虑  $T_1/T_s$  的值, 它被称为访问效率。这是平均访问时间 ( $T_s$ ) 与 M1 访问时间 ( $T_1$ ) 接近程度的度量。从式 (4.2) 可得:

$$\frac{T_1}{T_s} = \frac{1}{1 + (1-H) \frac{T_2}{T_1}} \quad (4.4)$$

图 4-22 绘制了  $T_1/T_s$  与命中率  $H$  的函数关系, 其中  $T_2/T_1$  是参数。通常, 片上高速缓存访问时间大约比主存访问时间快 25 ~ 50 倍 (即  $T_2/T_1$  为 25 ~ 50), 片外高速缓存访问时间大约比主存访问时间快 5 ~ 15 倍 (即  $T_2/T_1$  为 5 ~ 15), 主存访问时间大约比硬盘访问时间快 1000 倍 ( $T_2/T_1=1000$ )。因此, 为了满足性能要求, 似乎需要接近 0.9 范围内的命中率。

现在我们可以更准确地描述有关相对存储器大小的问题。对  $S_1 \ll S_2$ , 命中率为 0.8 或更好是合理的吗? 这将取决于许多因素, 包括执行软件的性质和两级存储设计的细节。当然主要的决定因素是局部性的程度。图 4-23 显示了局部性对命中率的影响。显然, 如果 M1 与 M2 大小相同, 那么命中率将为 1.0; M2 中所有的项也都存于 M1 中。现在假设没有局部性, 也就是说, 引用完全是随机的。在这种情况下, 命中率应该是相对存储器大小的严格线性函数。例如, 如果 M1 的大小是 M2 的一半, 那么任何时候 M2 中的一半项都存于 M1, 命中率将为 0.5。但实际上, 引用总是存在一定局部性的。中度和强局部性的影响如图所示。注意, 图 4-23 不是来自任何特定数据或模型, 它显示了不同程度局部性条件下的性能类型。

所以, 如果局部性很强, 哪怕是相对较小的上层存储器都可能获得较高的命中率。例如, 大量研究表明, 不论主存大小是多少, 相当小的高速缓存都会产生高于 0.75 的命中率 (如 [AGAR89]、[PRZY88]、[STRE83] 和 [SMIT82])。高速缓存大小在 1 K ~ 128 K 字范围内一般就足够了, 而主存现在通常是 GB 范围。当考虑虚拟存储器和硬盘缓存时, 我们将引

用其他研究来证实同样的现象，即由于局部性，相对较小的 M1 会产生较高命中率。

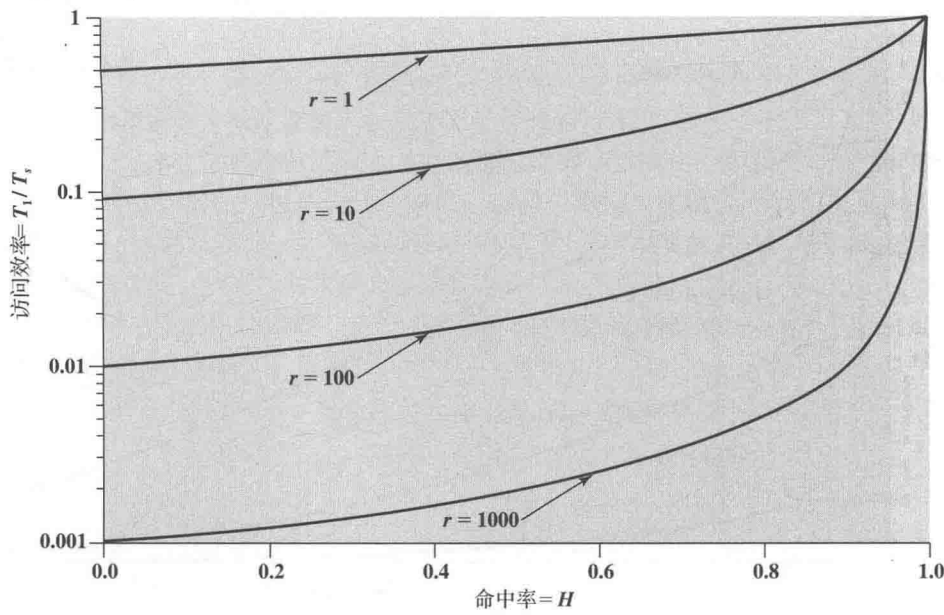


图 4-22    访问效率与命中率的函数关系 ( $r=T_2/T_1$ )

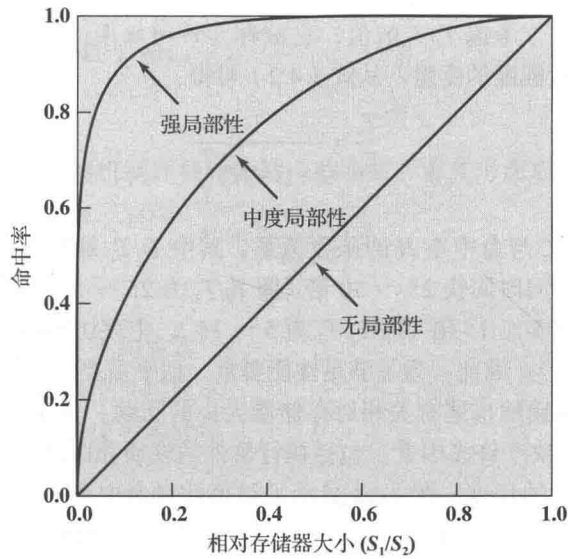


图 4-23    命中率与相对存储器大小的函数关系

这就引出了前面列出的最后一个问题：两个存储器的相对大小能满足成本需求吗？答案显然是肯定的。如果我们只需要一个相对较小的上层存储器来获得良好的性能，那么两级存储的每位平均成本就将接近于更便宜的下层存储器。

请注意，随着 L2 高速缓存，甚至是 L2 和 L3 高速缓存的出现，分析会变得相当复杂。相关讨论参见 [PEIR99] 和 [HAND98]。